

Unidade III

5 INTEGRAÇÃO REACT/NODE – CONSUMO DA API

A evolução de uma aplicação web frequentemente conduz à separação entre a experiência visual oferecida ao usuário e os serviços responsáveis por processar dados, aplicar regras e expor funcionalidades. Essa separação significa organizar responsabilidades em camadas capazes de evoluir com maior autonomia. De um lado, o front-end concentra a construção da interface, a interação com o usuário, a composição visual das telas e a resposta imediata aos eventos no navegador. De outro, a API concentra contratos HTTP, validações, persistência, regras de aplicação e integração com o banco de dados. A integração entre React, Node e uma API ASP.NET Core deve ser compreendida nesse cenário como uma arquitetura de colaboração entre partes distintas de uma mesma solução.

A React é uma biblioteca voltada à construção de interfaces baseadas em componentes. Em vez de tratar cada página como um documento estático, a interface é organizada em unidades reutilizáveis, capazes de receber dados, manter estado local e reagir a mudanças. Essa abordagem favorece experiências mais dinâmicas, nas quais filtros, listas, formulários, cartões, mensagens e elementos de navegação podem ser atualizados sem que toda a página precise ser reconstruída pelo servidor a cada interação. Em uma aplicação como o PetMatch, essa característica é útil para apresentar pets disponíveis, permitir refinamentos por espécie, porte ou localização, destacar informações relevantes e conduzir o usuário por uma jornada visual mais fluida.

Já o Node, nesse contexto, aparece como parte do ecossistema moderno de desenvolvimento front-end. Ele oferece a base de execução para ferramentas que organizam dependências, empacotam arquivos, executam servidores de desenvolvimento, processam módulos JavaScript e preparam a aplicação para publicação. Embora a API principal do sistema possa permanecer em ASP.NET Core, o ambiente Node contribui para a construção, o desenvolvimento e a organização do front-end React. Assim, a integração deve ser entendida como a combinação coordenada de plataformas especializadas em responsabilidades diferentes.

A API ASP.NET Core assume o papel de serviço de aplicação. Ela disponibiliza endpoints, recebe requisições, valida entradas, consulta ou altera dados persistidos e devolve respostas em formatos estruturados, usualmente JSON. O front-end React, por sua vez, consome esses endpoints para obter informações e enviar ações do usuário. Quando uma pessoa acessa uma lista de pets disponíveis, abre detalhes de um animal ou registra interesse em adoção, a interface não precisa conter toda a lógica de domínio. Ela deve solicitar dados à API, apresentar esses dados de modo compreensível e enviar ao servidor as informações necessárias para que a operação seja processada com segurança e consistência.

A experiência de uso pode ser aprimorada sem que regras centrais da aplicação sejam duplicadas no navegador. Da mesma forma, a API pode evoluir seus modelos, validações e mecanismos de persistência preservando contratos estáveis para os consumidores. Em sistemas reais, essa separação também permite que diferentes clientes consumam os mesmos serviços: uma aplicação React no navegador, um aplicativo móvel, uma interface administrativa ou uma integração institucional. O PetMatch, por tratar de adoção responsável, pode se beneficiar dessa estrutura ao manter uma fonte consistente de dados e regras, enquanto diferentes interfaces apresentam essas informações a públicos distintos.

O consumo de uma API pelo front-end exige atenção ao contrato publicado. A interface precisa conhecer os endereços dos endpoints, os métodos HTTP adequados, os formatos esperados de requisição e resposta, os códigos de erro possíveis e as regras de validação comunicadas pelo servidor. Não basta enviar e receber dados; é necessário interpretar corretamente os estados da comunicação. Uma lista vazia de pets, por exemplo, tem significado diferente de uma falha de conexão. Um erro de validação em um formulário de interesse de adoção precisa ser apresentado ao usuário de modo claro, enquanto uma indisponibilidade temporária do serviço exige outro tipo de mensagem. A qualidade da experiência depende da capacidade do front-end de traduzir respostas técnicas em orientações compreensíveis.

Também é necessário considerar a natureza assíncrona dessa integração. Requisições HTTP feitas pelo navegador não retornam instantaneamente em todos os cenários. A interface precisa lidar com carregamento, sucesso, erro, ausência de dados e atualização de estado. Em uma aplicação visualmente cuidadosa, esses estados não são detalhes secundários. Eles orientam a percepção do usuário sobre o funcionamento do sistema. No PetMatch, ao consultar pets disponíveis, a página deve indicar que os dados estão sendo carregados, apresentar os resultados com clareza, informar quando nenhum animal atende aos filtros selecionados e comunicar falhas sem comprometer a confiança na aplicação. A integração técnica, portanto, tem impacto direto na experiência humana.

A separação entre front-end e API também introduz questões de segurança e controle de acesso. Como parte do processamento ocorre no navegador, não se deve confiar exclusivamente na interface para preservar regras importantes. Validações no front-end podem melhorar a experiência, impedindo erros simples antes do envio, mas a decisão final deve pertencer à API. É no serviço de aplicação que as regras precisam ser confirmadas, pois requisições podem ser enviadas por outros clientes além da interface oficial. Esse princípio é essencial em qualquer sistema web e torna-se ainda mais importante quando a aplicação manipula informações de contato, registros de interesse e dados relacionados ao processo de adoção.

Uma equipe pode ajustar componentes visuais, reorganizar telas ou melhorar interações no React enquanto outra mantém endpoints, modelos de dados e regras no ASP.NET Core. Essa independência, porém, só é saudável quando existe coordenação por meio de contratos bem definidos. A documentação da API, os modelos de requisição e resposta e a padronização de erros funcionam como pontos de encontro entre as camadas. Sem esses acordos, a separação pode gerar desalinhamento; com eles, torna-se uma estratégia de organização e evolução.

No PetMatch, a integração entre React, Node e API deve preservar o sentido progressivo da aplicação. A interface não é um adorno sobre os dados, mas a forma pela qual pessoas interessadas em adotar compreendem possibilidades, avaliam responsabilidades e realizam ações. A API não é apenas um mecanismo técnico de entrega de JSON, mas a camada que protege a consistência do domínio e centraliza operações essenciais. Quando essas partes trabalham de modo coordenado, a aplicação consegue oferecer uma experiência rica no navegador sem perder disciplina arquitetural no servidor.

Este tópico introduz a integração entre front-end e API como uma separação coordenada entre experiência de uso e serviços de aplicação. O objetivo não é tratar React, Node e ASP.NET Core como ilhas tecnológicas, mas compreender como cada elemento contribui para uma solução web moderna. O front-end organiza a interação e a apresentação; o ambiente Node sustenta o desenvolvimento e a construção da interface; a API oferece contratos, processamento e acesso aos dados. Essa articulação permite que o PetMatch avance para uma experiência mais dinâmica e integrada, mantendo a responsabilidade técnica necessária para um sistema voltado à doação e adoção consciente de pets.



Saiba mais

Uma boa referência para React, com foco em componentes, Hooks, estado, roteamento, testes e integração com dados em aplicações web é:

BANKS, A.; PORCELLO, E. *Learning React: modern patterns for developing React apps*. 2. ed. Sebastopol: O'Reilly Media, 2020.

Já uma obra robusta para aprofundar Node.js, cobrindo programação assíncrona, streams, padrões de projeto, testes, escalabilidade e aplicações server-side em produção:

MAMMINO, L.; CASCIARO, M. *Node.js design patterns: level up your Node.js skills and design production-grade applications using proven techniques*. 4. ed. Birmingham: Packt Publishing, 2025.

5.1 Setup do front-end (Node LTS, Vite/Next), CORS e proxy de desenvolvimento

O objetivo principal deste tópico é compreender como uma aplicação web moderna pode ser dividida em duas partes que conversam entre si: um back-end, responsável pelas regras, dados e endpoints HTTP; e um front-end, responsável pela interface com a qual o usuário interage no navegador.



Lembrete

Em uma aplicação web tradicional, o servidor pode gerar páginas HTML completas e entregá-las prontas ao navegador. Esse modelo continua válido e é usado em muitas aplicações ASP.NET Core. Já em uma aplicação web moderna baseada em API, uma parte importante da interface pode ser executada no próprio navegador. Nesse modelo, o navegador carrega uma aplicação front-end, normalmente escrita em JavaScript ou TypeScript, e essa aplicação chama endpoints HTTP do back-end para buscar ou enviar dados.

O back-end, nesse cenário, não precisa montar toda a tela. Ele expõe dados e operações por meio de uma API. O front-end recebe esses dados e decide como apresentá-los ao usuário. No PetMatch, essa divisão fica como mostrado no quadro 7.

Quadro 7

Parte	Papel no sistema	Exemplo no PetMatch
Back-end	Mantém regras, banco de dados, endpoints HTTP, contratos OpenAPI e páginas ASP.NET existentes	Projeto <code>PetMatch.Web</code>
Front-end	Monta a interface React, chama a API, aplica filtros e renderiza cards no navegador	Projeto <code>PetMatch.Frontend</code> (figura 35)
API	Ponte de comunicação entre front-end e back-end	Endpoint <code>/api/pets</code>
Navegador	Executa a interface e exibe a experiência para o usuário	Home React do PetMatch

Essa separação ajuda a organizar responsabilidades. O back-end continua controlando os dados dos pets, a persistência em SQLite e os contratos HTTP. O front-end se concentra na experiência do usuário: filtros, cards, mensagens de carregamento, paginação e navegação para ações como cadastrar ou editar um pet.

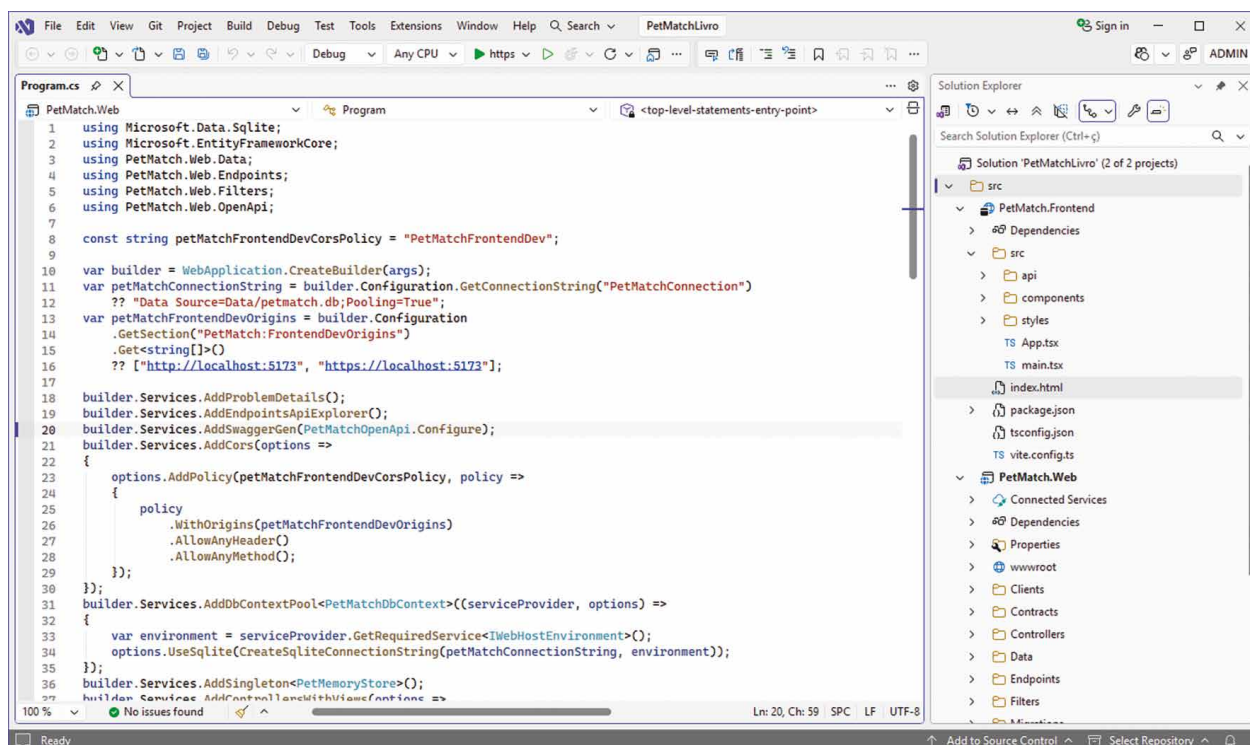


Figura 35 – Nova solução no Visual Studio 2026 com o projeto PetMatch.Frontend (à esquerda no Solution Manager)



Observação

A JavaScript nasceu como uma linguagem executada dentro do navegador. Com o tempo, surgiu a necessidade de executar JavaScript também fora do navegador, por exemplo para criar ferramentas de desenvolvimento, automatizar tarefas, compilar arquivos e iniciar servidores locais. O Node.js é um ambiente de execução de JavaScript fora do navegador. Ele permite que ferramentas do ecossistema front-end sejam executadas na máquina do desenvolvedor.

Neste tópico, Node.js não substitui o ASP.NET Core. O PetMatch continua usando ASP.NET Core no back-end. O papel do Node.js é apoiar o desenvolvimento do front-end: instalar pacotes, executar o Vite, compilar TypeScript e preparar os arquivos que o navegador vai carregar. Por isso usamos a versão LTS (Long Term Support ou Suporte de Longo Prazo) do Node.js. Em projetos didáticos e profissionais, a versão LTS costuma ser a escolha mais estável, pois recebe manutenção por mais tempo e evita problemas comuns de compatibilidade.

Ao instalar o Node.js, normalmente também instalamos o npm. O npm é o gerenciador de pacotes mais usado no ecossistema JavaScript. Ele baixa e organiza bibliotecas que o projeto precisa para funcionar. No front-end do PetMatch, o npm gerencia pacotes como mostrado no quadro 8.

Quadro 8

Pacote	Função
react	Biblioteca usada para construir interfaces baseadas em componentes
react-dom	Integra o React ao DOM do navegador
vite	Ferramenta de desenvolvimento e empacotamento do front-end
typescript	Permite escrever JavaScript com tipos estáticos
@vitejs/plugin-react	Integra o React ao Vite

Esses pacotes aparecem no arquivo `package.json`, o qual funciona como uma espécie de manifesto do projeto front-end. Ele descreve dependências, scripts e comandos usados durante o desenvolvimento.

O React é uma biblioteca JavaScript para construir interfaces de usuário. Em vez de escrever uma página inteira como um único bloco de HTML, o desenvolvedor divide a interface em componentes menores. Um componente é uma parte reutilizável da tela. Ele pode representar um botão, um card, uma barra de filtros, uma lista, um cabeçalho ou uma página inteira. No PetMatch, por exemplo, a home React pode ser dividida em componentes como demonstrado no quadro 9.

Quadro 9

Componente	Responsabilidade
Hero	Apresenta a chamada principal da tela e ações como cadastrar ou gerenciar pets
PetFilters	Permite filtrar por espécie, porte e cidade
PetCard	Exibe os dados principais de um pet
Pagination	Controla a navegação entre páginas de resultados
App	Coordena estado, chamadas à API e composição da tela

Essa divisão deixa a interface mais fácil de entender. Em vez de procurar tudo em um arquivo enorme, o leitor pode estudar cada parte separadamente. A React também trabalha com estado. Estado é a memória momentânea da interface. No PetMatch, a página atual, os filtros escolhidos, a lista de pets carregada, a mensagem de erro e o indicador de carregamento são exemplos de estado. Quando o estado muda, a React atualiza a tela.

Já a TypeScript é uma extensão de JavaScript que adiciona tipos. Um tipo descreve o formato esperado de um valor. Em JavaScript puro, uma função pode receber qualquer objeto, mesmo que ele não tenha as propriedades esperadas. Em TypeScript, podemos declarar que um pet recebido da API deve ter campos como id, nome, especie, porte, cidade e descricao.



Lembrete

No PetMatch, isso ajuda o leitor a enxergar o contrato entre back-end e front-end. Quando a API retorna uma lista de pets, o front-end espera um formato específico. A TypeScript torna esse formato explícito no código.

O Vite é uma ferramenta para desenvolver e empacotar aplicações front-end. Durante o desenvolvimento, ele inicia um servidor local rápido. Esse servidor entrega os arquivos da aplicação React ao navegador e atualiza a tela quando o código muda. Sem uma ferramenta como o Vite, o fluxo de desenvolvimento seria mais trabalhoso. O desenvolvedor teria que lidar manualmente com compilação, organização de módulos, atualização do navegador e preparação dos arquivos finais.

No PetMatch, o Vite será executado na porta 5173. Assim, durante o desenvolvimento, a interface React ficará disponível em `http://localhost:5173`. O back-end ASP.NET Core continuará em sua própria porta, por exemplo: `http://localhost:5213`. Essa diferença de portas é importante para entender CORS e proxy.

Para o navegador, uma origem é formada por protocolo, domínio e porta. Assim, estes dois endereços são origens diferentes: `http://localhost:5173` e `http://localhost:5213`. Embora ambos estejam na mesma máquina, as portas são diferentes. Para o navegador, isso já basta para tratar as duas URLs como origens distintas. Essa regra existe por segurança. Uma página carregada de uma origem não pode simplesmente acessar qualquer recurso de outra origem sem permissão. O mecanismo que controla essa permissão se chama CORS, que significa Cross-Origin Resource Sharing. Em português, podemos entender como compartilhamento de recursos entre origens diferentes.

Quando a interface React do PetMatch executa em `http://localhost:5173` e chama a API ASP.NET Core em `http://localhost:5213`, o navegador verifica se o back-end permite essa comunicação. Se o back-end não declarar essa permissão, a chamada pode ser bloqueada pelo navegador.



Observação

O CORS não é um sistema de login, pois não substitui autenticação nem autorização, apenas diz ao navegador quais origens podem chamar a API. É uma aplicação ainda precisa de seus próprios mecanismos de segurança para proteger dados sensíveis e ações restritas. No PetMatch, a política CORS será usada no ambiente de desenvolvimento para permitir que a interface React chame a API local.

Um proxy é um intermediário. No contexto deste tópico, o proxy de desenvolvimento permite que chamadas feitas pela interface React sejam encaminhadas ao back-end ASP.NET Core. O front-end pode chamar `/api/pets` em vez de escrever a URL completa do back-end dentro do código da

tela, o Vite recebe essa chamada e a encaminha para `http://localhost:5213/api/pets` e isso simplifica o desenvolvimento. O código React fica mais limpo e o projeto consegue abrir também rotas ASP.NET existentes, como `/Pets/Create` e `/Pets/Edit/1`, passando pela porta do Vite.

O React pode ser usado com diferentes ferramentas. Duas escolhas conhecidas são Vite e Next.js. O Next.js é um framework completo. Ele oferece roteamento próprio, renderização no servidor, convenções de projeto e recursos avançados para aplicações React. Ele é muito útil quando a aplicação precisa dessas características desde o início. O Vite, por outro lado, é mais direto para um primeiro front-end React que consome uma API existente. Como o PetMatch já possui um back-end ASP.NET Core com endpoints, páginas e banco de dados, o objetivo deste subtópico é criar uma interface cliente leve que chame `/api/pets`, exiba cards, aplique filtros e navegue para rotas já existentes. Por isso, Vite é uma boa escolha didática aqui. Ele permite estudar React, TypeScript, chamadas HTTP, CORS e proxy sem introduzir muitas convenções novas ao mesmo tempo.

A solução passa a conter dois projetos principais conforme apontado no quadro 10.

Quadro 10

Projeto	Responsabilidade
PetMatch.Web	Back-end ASP.NET Core, API, páginas existentes, banco SQLite, OpenAPI e configurações de CORS
PetMatch.Frontend	Front-end React com TypeScript, Vite, componentes e estilos da nova home

O projeto `PetMatch.Web` continua sendo o ponto de partida da aplicação. Ele preserva a API, os endpoints, a documentação OpenAPI, o banco SQLite e as páginas ASP.NET. O projeto `PetMatch.Frontend` concentra a experiência React. Ele possui arquivos como os demonstrados no quadro 11.

Quadro 11

Arquivo ou pasta	Função
<code>PetMatch.Frontend.esproj</code>	Integra o projeto JavaScript/TypeScript ao Visual Studio
<code>package.json</code>	Lista dependências e scripts npm
<code>vite.config.ts</code>	Configura o Vite, a porta e o proxy de desenvolvimento
<code>index.html</code>	Página HTML base que carrega a aplicação React
<code>src/api</code>	Código de comunicação com a API
<code>src/components</code>	Componentes React reutilizáveis
<code>src/styles</code>	Estilos visuais da interface

No projeto `PetMatch.Frontend`, o arquivo `PetMatch.Frontend.esproj` usa o SDK JavaScript do Visual Studio:

```
<Project Sdk="Microsoft.VisualStudio.JavaScript.Sdk">
  <PropertyGroup>
```



```
<StartupCommand>npm run dev</StartupCommand>
<JavaScriptTestRoot>src</JavaScriptTestRoot>
<ShouldRunNpmInstall>true</ShouldRunNpmInstall>
</PropertyGroup>
</Project>
```

Esse arquivo não representa uma página da aplicação, mas serve para que o Visual Studio reconheça o projeto front-end e saiba como trabalhar com ele. A propriedade `StartupCommand` indica o comando usado para iniciar o servidor de desenvolvimento. O comando `npm run dev` normalmente aciona o Vite. A propriedade `ShouldRunNpmInstall` permite que o Visual Studio restaure dependências JavaScript quando necessário.

A ideia central é esta: o arquivo `.esproj` ajuda a IDE (Integrated Development Environment ou Ambiente de Desenvolvimento Integrado) a tratar o front-end como parte da solução, mesmo que ele use ferramentas diferentes do back-end ASP.NET Core. O fluxo de execução do PetMatch usa o projeto ASP.NET Core como ponto de partida. O leitor inicia `PetMatch.Web` no Visual Studio, e o SPA (Single Page Application) proxy aciona o servidor Vite na pasta do front-end. No arquivo `PetMatch.Web.csproj`, a configuração do SPA proxy fica assim:

```
<PropertyGroup>
  <SpaRoot>..\PetMatch.Frontend</SpaRoot>
  <SpaProxyServerUrl>http://localhost:5173</SpaProxyServerUrl>
  <SpaProxyLaunchCommand>npm run dev</SpaProxyLaunchCommand>
</PropertyGroup>
```

Cada propriedade tem uma função de acordo com o mostrado no quadro 12.

Quadro 12

Propriedade	Significado
<code>SpaRoot</code>	Indica onde está o projeto front-end
<code>SpaProxyServerUrl</code>	Informa a URL do servidor Vite
<code>SpaProxyLaunchCommand</code>	Define o comando usado para iniciar o front-end

Com essa configuração, o leitor não precisa iniciar manualmente dois processos separados no primeiro contato com a aplicação. Ao executar o back-end, o Visual Studio consegue acionar também o fluxo do front-end. Como a interface React executa em `http://localhost:5173` e o back-end ASP.NET Core executa em outra porta, o back-end precisa permitir essa origem durante o desenvolvimento. No arquivo `Program.cs`, criamos uma política CORS:

```
const string frontendDevelopmentPolicy = "PetMatchFrontendDev";

builder.Services.AddCors(options =>
{
    options.AddPolicy(frontendDevelopmentPolicy, policy =>
    {
        policy
```

```
        .WithOrigins("http://localhost:5173")  
        .AllowAnyHeader()  
        .AllowAnyMethod();  
    });  
});
```

Esse trecho registra uma política chamada `PetMatchFrontendDev`. Ela permite chamadas vindas da origem do Vite. Também permite cabeçalhos e métodos HTTP necessários para as chamadas feitas pela interface. Depois de registrar a política, ela precisa ser aplicada ao pipeline da aplicação:

```
if (app.Environment.IsDevelopment())  
{  
    app.UseCors(frontendDevelopmentPolicy);  
}
```

Neste momento, queremos permitir a comunicação entre as portas locais usadas no estudo. Em ambiente de produção, as origens permitidas devem ser avaliadas com mais cuidado. O arquivo `vite.config.ts`, no projeto `PetMatch.Frontend`, configura o servidor de desenvolvimento do Vite:

```
import { defineConfig } from 'vite';  
import react from '@vitejs/plugin-react';  
  
const backendUrl = 'http://localhost:5213';  
  
export default defineConfig({  
  plugins: [react()],  
  server: {  
    port: 5173,  
    proxy: {  
      '/api': {  
        target: backendUrl,  
        changeOrigin: true,  
        secure: false  
      },  
      '/Pets': {  
        target: backendUrl,  
        changeOrigin: true,  
        secure: false  
      },  
      '/css': {  
        target: backendUrl,  
        changeOrigin: true,  
        secure: false  
      },  
      '/js': {  
        target: backendUrl,  
        changeOrigin: true,  
        secure: false  
      }  
    }  
  }  
});
```

A propriedade `port` define a porta do Vite. A propriedade `proxy` define quais caminhos devem ser encaminhados para o back-end. No PetMatch, quatro grupos de caminhos são importantes, conforme quadro 13.

Quadro 13

Caminho	Por que encaminhar
<code>/api</code>	Permite que a home React chame os endpoints da API
<code>/Pets</code>	Permite abrir páginas ASP.NET de cadastro, listagem e edição
<code>/css</code>	Permite carregar estilos das páginas ASP.NET quando abertas pela porta do Vite
<code>/js</code>	Permite carregar scripts das páginas ASP.NET quando abertas pela porta do Vite

Com isso, a home React pode chamar `/api/pets?page=1&pageSize=4` e o Vite encaminha a requisição ao back-end. Da mesma forma, links como `/Pets/Create` e `/Pets/Edit/1` continuam funcionando durante o desenvolvimento.

Uma interface React precisa buscar dados em algum lugar. No PetMatch, os dados dos pets vêm da API do back-end. Para não espalhar chamadas `fetch` por vários componentes, criamos um pequeno cliente em `src/api/petmatchClient.ts`. Esse arquivo concentra o formato dos dados e a função de busca.

```
export type PetResponse = {
  id: number;
  nome: string;
  especie: string;
  porte: string;
  idadeAproximada: number;
  cidade: string;
  descricao: string;
  icone: string;
  disponivel: boolean;
  caracteristicas: string[];
};

export type PagedResponse<T> = {
  items: T[];
  page: number;
  pageSize: number;
  totalItems: number;
  totalPages: number;
};

export type PetQuery = {
  page: number;
  pageSize: number;
  especie?: string;
  porte?: string;
  cidade?: string;
};
```

```
export async function listarPets(query: PetQuery):  
Promise<PagedResponse<PetResponse>> {  
  const params = new URLSearchParams();  
  
  params.set('page', String(query.page));  
  params.set('pageSize', String(query.pageSize));  
  
  if (query.especie) {  
    params.set('especie', query.especie);  
  }  
  
  if (query.porte) {  
    params.set('porte', query.porte);  
  }  
  
  if (query.cidade) {  
    params.set('cidade', query.cidade);  
  }  
  
  const response = await fetch(`/api/pets?${params.toString()}`);  
  
  if (!response.ok) {  
    throw new Error('Não foi possível carregar os pets agora.');  }  
  
  return response.json();  
}
```

`PetResponse` descreve um pet vindo da API. `PagedResponse<T>` descreve uma resposta paginada, isto é, uma resposta que não traz todos os registros de uma vez, mas apenas uma página de resultados. `PetQuery` descreve os filtros enviados para a API.

A função `listarPets` monta os parâmetros da URL com `URLSearchParams`. Se o usuário escolheu espécie, porte ou cidade, esses filtros entram na consulta. Em seguida, a função chama `/api/pets` com `fetch`. O uso de `/api/pets`, sem escrever `http://localhost:5213`, é possível por causa do proxy configurado no Vite.

Antes de olharmos o componente principal, vale entender três ideias do React.

- **Estado:** memória da tela. No `PetMatch`, a página atual, a lista de pets, o filtro de espécie, o filtro de porte, o filtro de cidade, o carregamento e o erro são estados.
- **Efeito:** ação que acontece como consequência de uma mudança. Buscar dados na API é um exemplo de efeito. Quando o usuário muda um filtro, a tela precisa buscar novamente os pets.
- **Renderização:** processo de transformar o estado atual em elementos visuais. Se a lista de pets muda, a grade de cards também muda.

O componente `App`, em `src/App.tsx`, coordena estas partes:

```
import { useEffect, useState } from 'react';  
import { listarPets, PetResponse } from './api/petmatchClient';  
import { Hero } from './components/Hero';
```

```
import { PetCard } from './components/PetCard';
import { PetFilters } from './components/PetFilters';
import { Pagination } from './components/Pagination';

export function App() {
  const [pets, setPets] = useState<PetResponse[]>([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [especie, setEspecie] = useState('');
  const [porte, setPorte] = useState('');
  const [cidade, setCidade] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    setLoading(true);
    setError('');

    listarPets({ page, pageSize: 4, especie, porte, cidade })
      .then(result => {
        setPets(result.items);
        setTotalPages(result.totalPages);
      })
      .catch(() => setError('Não foi possível carregar os pets agora. Tente novamente em instantes.'))
      .finally(() => setLoading(false));
  }, [page, especie, porte, cidade]);

  return (
    <main className="app-shell">
      <Hero totalPets={pets.length} />

      <PetFilters
        especie={especie}
        porte={porte}
        cidade={cidade}
        onEspecieChange={setEspecie}
        onPorteChange={setPorte}
        onCidadeChange={setCidade}
        onPageReset={() => setPage(1)}
      />

      {loading && <p className="status-card">Buscando pets disponíveis...</p>}
      {error && <p className="status-card status-error">{error}</p>}

      <section className="pet-grid">
        {pets.map(pet => <PetCard key={pet.id} pet={pet} />)}
      </section>

      <Pagination page={page} totalPages={totalPages} onPageChange={setPage} />
    </main>
  );
}
```

O `useState` cria valores que podem mudar com o tempo. Por exemplo, `page` guarda a página atual e `setPage` altera essa página. O mesmo vale para filtros, lista de pets, carregamento e erro.

O `useEffect` executa a busca na API. Ele depende de `page`, `espécie`, `porte` e `cidade`. Isso significa que, quando qualquer um desses valores muda, o React executa novamente a busca.

A parte final do componente descreve a interface: primeiro o `hero`, depois os filtros, depois mensagens de estado, cards e paginação.

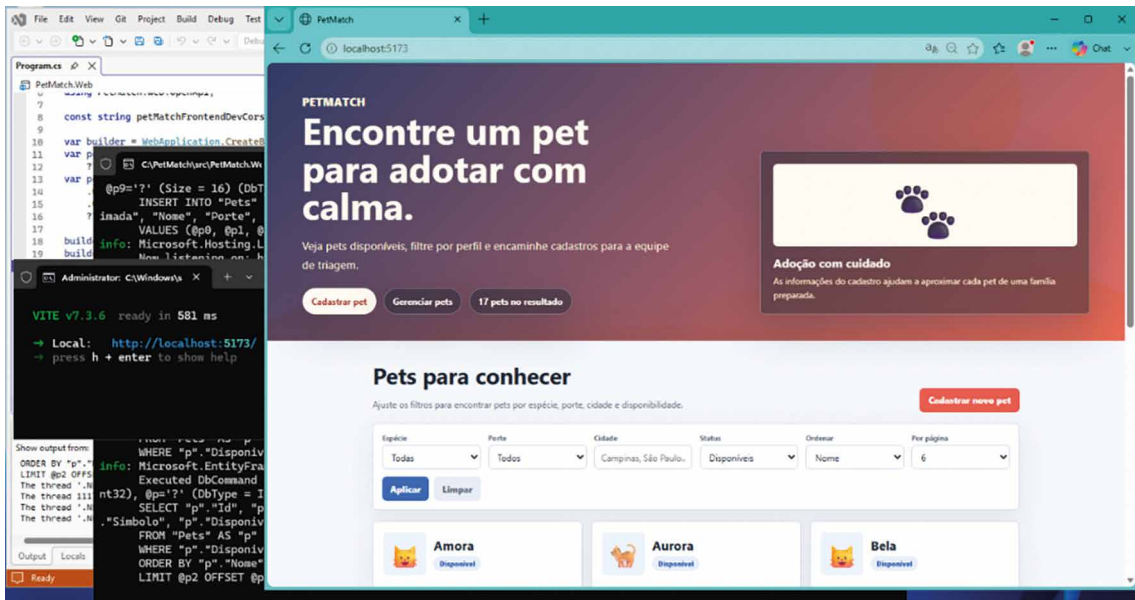


Figura 36 – Página home do PetMatch aberta em `http://localhost:5213`. A paleta de cores foi alterada para diferenciar a interface React da interface MVC do ASP.NET, que continua existindo no PetMatch. Note também à esquerda na porção intermediária a execução no console do servidor Vite pelo Visual Studio

O componente `Hero`, em `src/components/Hero.tsx`, apresenta a proposta da tela e oferece ações diretas:

```
type HeroProps = {
  totalPets: number;
};

export function Hero({ totalPets }: HeroProps) {
  return (
    <section className="hero">
      <div>
        <p className="eyebrow">Adoção responsável</p>
        <h1>Encontre um pet para fazer parte da sua rotina.</h1>
        <p>
          Conheça animais disponíveis, filtre por perfil e avance para o cadastro
          ou gerenciamento quando encontrar uma boa combinação.
        </p>
        <div className="hero-actions">
          <a href="/Pets/Create" className="primary-action">Cadastrar pet</a>
          <a href="/Pets" className="secondary-action">Gerenciar pets</a>
        </div>
      </div>
    </section>
  );
}
```

```

    <div className="hero-card">
      <strong>{totalPets}</strong>
      <span>pets exibidos nesta busca</span>
    </div>
  </section>
);
}

```

Esse componente mostra que a interface React não está isolada do restante do sistema. Os links apontam para rotas ASP.NET já existentes. O funcionamento desses links pela porta do Vite depende do proxy configurado para `/Pets`.

Cada pet precisa ser apresentado de forma clara. O componente `PetCard`, em `src/components/PetCard.tsx`, recebe um pet e renderiza seus dados principais:

```

import { PetResponse } from '../api/petmatchClient';

type PetCardProps = {
  pet: PetResponse;
};

export function PetCard({ pet }: PetCardProps) {
  return (
    <article className="pet-card">
      <div className="pet-icon">{pet.icone}</div>

      <div className="pet-card-content">
        <h2>{pet.nome}</h2>
        <p>{pet.descricao}</p>

        <div className="pet-meta">
          <span>{pet.especie}</span>
          <span>{pet.porte}</span>
          <span>{pet.idadeAproximada} ano(s)</span>
          <span>{pet.cidade}</span>
        </div>

        <div className="pet-tags">
          {pet.caracteristicas.map(caracteristica => (
            <span key={caracteristica}>{caracteristica}</span>
          ))}
        </div>

        <a className="card-action" href={` /Pets/Edit/${pet.id} `}>
          Editar cadastro
        </a>
      </div>
    </article>
  );
}

```

O `PetCard` não precisa saber como buscar dados, nem como funciona a paginação, nem como os filtros foram escolhidos. Ele recebe um objeto pet e exibe esse objeto na tela. O link Editar cadastro abre uma página ASP.NET existente. Assim, o front-end React e as páginas tradicionais convivem na mesma aplicação durante esta etapa do livro.

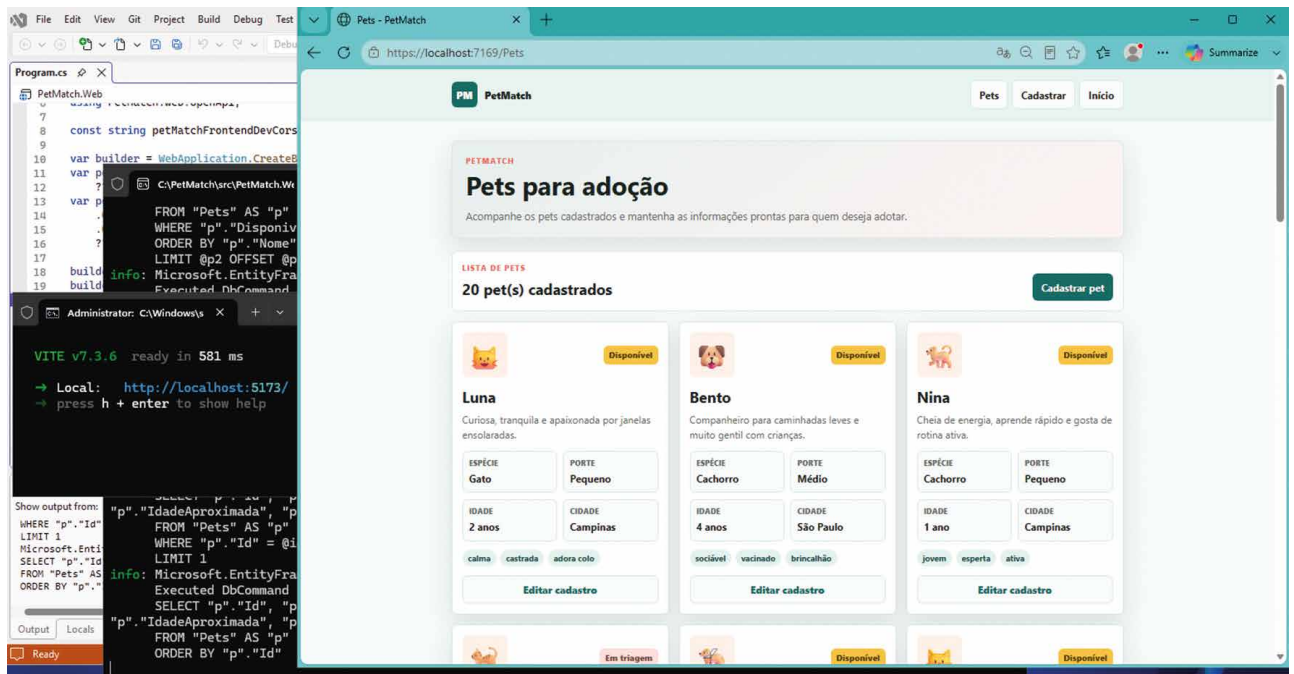


Figura 37 – Página ASP.NET em <http://localhost:7169/Pets> para a lista de pets. A paleta de cores original foi mantida para destacar visualmente a coexistência do projeto React na página home (figura 36)

Os estilos da nova home ficam em `src/styles/petmatch.css`. O objetivo não é apenas deixar a tela mais bonita. O estilo também organiza hierarquia visual, espaçamento, leitura e responsividade.

```
.app-shell {
  min-height: 100vh;
  padding: 28px clamp(16px, 4vw, 48px);
  background: #fff7ed;
  color: #17324d;
}

.hero {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 220px;
  gap: 24px;
  align-items: center;
  margin: 0 auto 24px;
  max-width: 1120px;
}

.hero-actions {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  margin-top: 18px;
}
```

```
.primary-action {
  background: #ff6b57;
  color: #fff;
}

.secondary-action {
  background: #17324d;
  color: #fff;
}

.pet-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 18px;
  margin: 24px auto 0;
  max-width: 1120px;
}

.pet-card {
  background: #ffffff;
  border: 1px solid #f4c95d;
  border-radius: 18px;
  padding: 18px;
  box-shadow: 0 18px 40px rgba(23, 50, 77, 0.1);
}

.status-card {
  max-width: 1120px;
  margin: 18px auto 0;
  padding: 16px;
  border-radius: 14px;
  background: #eaf4ff;
}

.status-error {
  background: #fff0ed;
  color: #9f2d20;
}
```

A propriedade `display: grid` ajuda a organizar áreas da tela. Em `.pet-grid`, a função `repeat(auto-fit, minmax(240px, 1fr))` permite que os cards se ajustem ao espaço disponível. Isso torna a tela mais flexível em diferentes tamanhos de janela.

A classe `.status-card` cria uma área visual para mensagens de carregamento, erro ou ausência de resultados. Isso é importante porque uma boa interface não mostra apenas o resultado ideal. Ela também precisa orientar o usuário quando algo está carregando ou quando a API está temporariamente indisponível.

Quando o usuário abre a home React do PetMatch, o navegador carrega a aplicação servida pelo Vite. O componente `App` inicia com estado de carregamento. O `useEffect` chama `listarPets`. A função `listarPets` faz uma requisição para `/api/pets`. O Vite encaminha essa requisição ao back-end ASP.NET Core. A API consulta os dados e devolve uma resposta paginada. O React recebe os dados e renderiza os cards. A figura 38 apresenta esse fluxo.

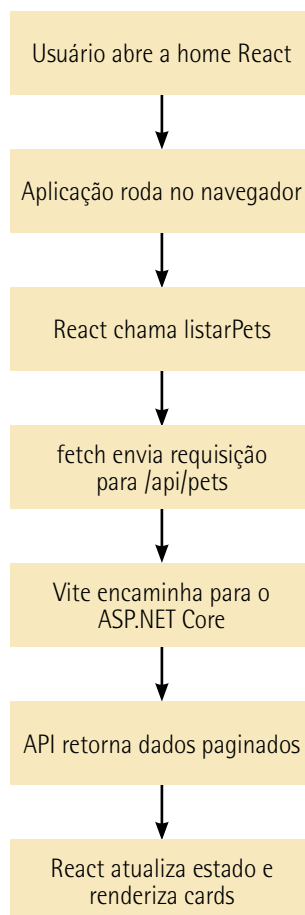


Figura 38 – Resumo das ações na página home

O ponto importante é perceber que a tela não contém os dados fixos dos pets. Ela busca esses dados no back-end. Por isso a interface reflete os dados reais da aplicação. Os detalhes práticos do PetMatch continuam presentes de acordo com o quadro 14.

Quadro 14

Recurso	Como aparece na implementação
API de pets	Consumida por <code>/api/pets</code>
Paginação	Controlada por <code>page</code> , <code>pageSize</code> , <code>totalItems</code> e <code>totalPages</code>
Filtros	Enviados por espécie, porte e cidade
Cadastro	Acessado pelo link <code>/Pets/Create</code>
Gerenciamento	Acessado pelo link <code>/Pets</code>
Edição	Acessada por <code>/Pets/Edit/{id}</code>
Estilos ASP.NET existentes	Encaminhados pelo proxy de <code>/css</code> e <code>/js</code>

5.2 Fetch/axios, estados de requisição, roteamento e páginas de lista/detalhe

Uma interface web moderna raramente apresenta apenas conteúdo escrito diretamente no HTML. Em aplicações conectadas a uma API, grande parte da tela nasce de uma requisição HTTP feita pelo navegador, de uma resposta recebida em JSON e de uma atualização do estado visual da aplicação. Esse fluxo precisa ser tratado como parte central da experiência do usuário. Ao solicitar dados remotos, a página passa por momentos diferentes: antes da requisição, durante o carregamento, depois de uma resposta bem-sucedida, diante de uma lista vazia ou quando ocorre uma falha. Uma aplicação bem construída representa esses estados de forma clara, porque o usuário precisa entender o que está acontecendo enquanto aguarda os dados.

No front-end, `fetch` é a API nativa do navegador para realizar requisições HTTP. Ela permite chamar um endpoint, aguardar a resposta, verificar o status HTTP e converter o corpo para JSON quando a resposta traz dados estruturados. `Axios` é uma biblioteca bastante usada no ecossistema JavaScript para o mesmo tipo de tarefa, oferecendo conveniências como configuração centralizada, interceptadores e tratamento uniforme de respostas. Neste subtópico, a implementação usa `fetch`, pois o PetMatch já possui um cliente HTTP simples, suficiente para tornar visível o percurso da requisição: montar parâmetros, chamar a API, verificar sucesso ou falha, converter o JSON e entregar os dados aos componentes React.

O consumo HTTP deve ser organizado fora dos componentes visuais sempre que possível. Quando a chamada à API fica misturada com a marcação da tela, a leitura do componente se torna mais difícil e a repetição cresce. Um pequeno cliente HTTP, mesmo simples, cria uma fronteira clara: os componentes pedem dados em termos do domínio da aplicação, enquanto o cliente conhece os detalhes do endpoint. No caso de uma listagem, ele sabe chamar `/api/pets` com paginação e filtros; no caso de um detalhe, sabe chamar `/api/pets/{id}`. Essa separação prepara a interface para evoluir sem espalhar URLs e regras de tratamento de resposta por toda a aplicação.

Os estados de requisição são uma forma de modelar a conversa entre a tela e a API. O estado de carregamento evita que a página pareça vazia enquanto a chamada está em andamento. O estado de sucesso apresenta os dados recebidos. O estado vazio comunica que a requisição funcionou, mas não há itens compatíveis com os filtros. O estado de erro comunica que a resposta não pôde ser usada ou que o recurso solicitado não existe. Esses estados definem a confiabilidade percebida da aplicação. Uma tela que não distingue carregamento de falha deixa o usuário sem orientação.

O roteamento do front-end organiza as páginas renderizadas no navegador. Em uma aplicação React, a URL pode determinar qual componente de página será apresentado, sem depender de uma navegação completa no servidor para cada tela da experiência cliente. A rota `/` pode apresentar a listagem; a rota `/pets/:id` pode apresentar o detalhe de um pet. Esse roteamento é diferente do roteamento da API: a API resolve endpoints como `/api/pets/2`, enquanto o React resolve páginas como `/pets/2`. A distinção permite que a aplicação tenha uma experiência de navegação fluida e, ao mesmo tempo, continue consumindo contratos HTTP do back-end.

O padrão lista/detalhe é uma das formas mais comuns de organizar aplicações de consulta. A lista oferece visão geral, filtros, paginação e comparação rápida entre itens. O detalhe concentra informações de um item selecionado, com mais espaço para descrição, características e ações relacionadas. Para o usuário, esse padrão é natural: primeiro ele encontra um pet que desperta interesse; depois abre uma página específica para ler com mais atenção. Para o código, esse padrão também é saudável, pois a listagem e o detalhe possuem responsabilidades diferentes e podem ser implementados como páginas React separadas.

No PetMatch, o front-end já possui Vite, proxy de desenvolvimento, CORS configurado, hero, filtros, cards e paginação. A evolução deste tópico cria uma estrutura de páginas em `PetMatch.Frontend: PetListPage.tsx` para a listagem e `PetDetailsPage.tsx` para o detalhe. Também são criados componentes auxiliares para estados visuais, como `RequestStatus.tsx` e `EmptyState.tsx`. O objetivo é manter a experiência do usuário voltada à adoção responsável, sem telas técnicas, painéis de API ou simuladores. A navegação principal passa a permitir que o usuário veja pets, filtre resultados, abra detalhes e retorne à lista.

Para usar roteamento no React, o projeto `PetMatch.Frontend` adiciona a dependência `npm react-router-dom` na versão 7.18.1. O arquivo `package.json`, no projeto `PetMatch.Frontend`, passa a incluir a dependência de roteamento, restaurada pelo suporte JavaScript do Visual Studio 2026.

```
{
  "dependencies": {
    "react-router-dom": "7.18.1"
  }
}
```

Esse trecho registra a biblioteca que permite declarar rotas e ler parâmetros da URL dentro dos componentes React. Após salvar o arquivo no Visual Studio, as dependências `npm` são restauradas pela própria IDE. A partir disso, os componentes podem usar `BrowserRouter`, `Routes`, `Route`, `Link` e `useParams`.

No projeto `PetMatch.Frontend`, o arquivo `src/main.tsx` envolve a aplicação com `BrowserRouter`. Esse componente fornece ao React Router o contexto necessário para interpretar a URL do navegador e renderizar a página correspondente.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { App } from './App';
import './styles/petmatch.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

O CSS global continua sendo importado no ponto de entrada e `App` passa a receber o contexto de roteamento. Sem essa configuração, as páginas React não teriam acesso ao mecanismo que associa URL e componente. O arquivo `src/App.tsx`, no projeto `PetMatch.Frontend`, deixa de concentrar a tela inteira e passa a declarar as rotas principais da interface. A rota inicial exibe a listagem; a rota parametrizada exibe o detalhe de um pet.

```
import { Route, Routes } from 'react-router-dom';
import { PetListPage } from '../pages/PetListPage';
import { PetDetailsPage } from '../pages/PetDetailsPage';

export function App() {
  return (
    <Routes>
      <Route path="/" element={<PetListPage />} />
      <Route path="/pets/:id" element={<PetDetailsPage />} />
    </Routes>
  );
}
```

Esse arquivo mostra a mudança de organização da interface. A aplicação deixa de ser uma tela única e passa a ter páginas. A rota `/pets/:id` usa um parâmetro chamado `id`, que será lido pela página de detalhe para buscar o pet correspondente na API.

O cliente HTTP também evolui. No projeto `PetMatch.Frontend`, o arquivo `src/api/petmatchClient.ts` preserva `listarPets` e adiciona `obterPet`, usando `fetch` para consultar `/api/pets/{id}`. O mesmo arquivo mantém os tipos `PetResponse`, `PagedResponse<T>` e `PetQuery`.

```
export async function obterPet(id: number): Promise<PetResponse> {
  const response = await fetch(`/api/pets/${id}`);

  if (!response.ok) {
    throw new Error('Não foi possível carregar os detalhes deste pet.');
```

Essa função concentra a chamada de detalhe em um único ponto. A página React não precisa montar manualmente o endpoint nem decidir como interpretar uma resposta sem sucesso. Ela chama `obterPet`, aguarda o resultado e atualiza seus estados visuais conforme a resposta.

A listagem fica em `src/pages/PetListPage.tsx`, no projeto `PetMatch.Frontend`. Essa página concentra filtros, paginação, carregamento, erro e estado vazio. O trecho a seguir mostra a estrutura essencial: a página chama `listarPets`, atualiza os itens recebidos e decide qual estado visual apresentar.

```
import { useEffect, useState } from 'react';
import { listarPets, PetResponse } from '../api/petmatchClient';
import { EmptyState } from '../components/EmptyState';
import { Hero } from '../components/Hero';
import { Pagination } from '../components/Pagination';
import { PetCard } from '../components/PetCard';
import { PetFilters } from '../components/PetFilters';
import { RequestStatus } from '../components/RequestStatus';

export function PetListPage() {
  const [pets, setPets] = useState<PetResponse[]>([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [especie, setEspecie] = useState('');
  const [porte, setPorte] = useState('');
  const [cidade, setCidade] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    setLoading(true);
    setError('');

    listarPets({ page, pageSize: 4, especie, porte, cidade })
      .then(result => {
        setPets(result.items);
        setTotalPages(result.totalPages);
      })
      .catch(() => setError('Não foi possível carregar os pets agora.'))
      .finally(() => setLoading(false));
  }, [page, especie, porte, cidade]);

  return (
    <main className="app-shell">
      <Hero totalPets={pets.length} />

      <PetFilters
        especie={especie}
        porte={porte}
        cidade={cidade}
        onEspecieChange={setEspecie}
        onPorteChange={setPorte}
        onCidadeChange={setCidade}
        onPageReset={() => setPage(1)}
      />

      {loading && <RequestStatus title="Buscando pets" description="Estamos carregando os pets disponíveis." />}
      {error && <RequestStatus tone="error" title="Não foi possível carregar" description={error} />}

      {!loading && !error && pets.length === 0 && (
        <EmptyState title="Nenhum pet encontrado" description="Ajuste os filtros para ver outras possibilidades." />
      )}
    </main>
  );
}
```

```

<section className="pet-grid">
  {pets.map(pet => <PetCard key={pet.id} pet={pet} />)}
</section>

<Pagination page={page} totalPages={totalPages} onPageChange={setPage} />
</main>
);
}

```

Esse código torna explícitos os estados da requisição. Enquanto `loading` está ativo, a página informa que está buscando dados. Quando ocorre erro, a mensagem é exibida em um componente visual próprio. Quando a chamada funciona e a lista vem vazia, o usuário recebe orientação para ajustar os filtros. Quando há dados, os cards são renderizados normalmente.

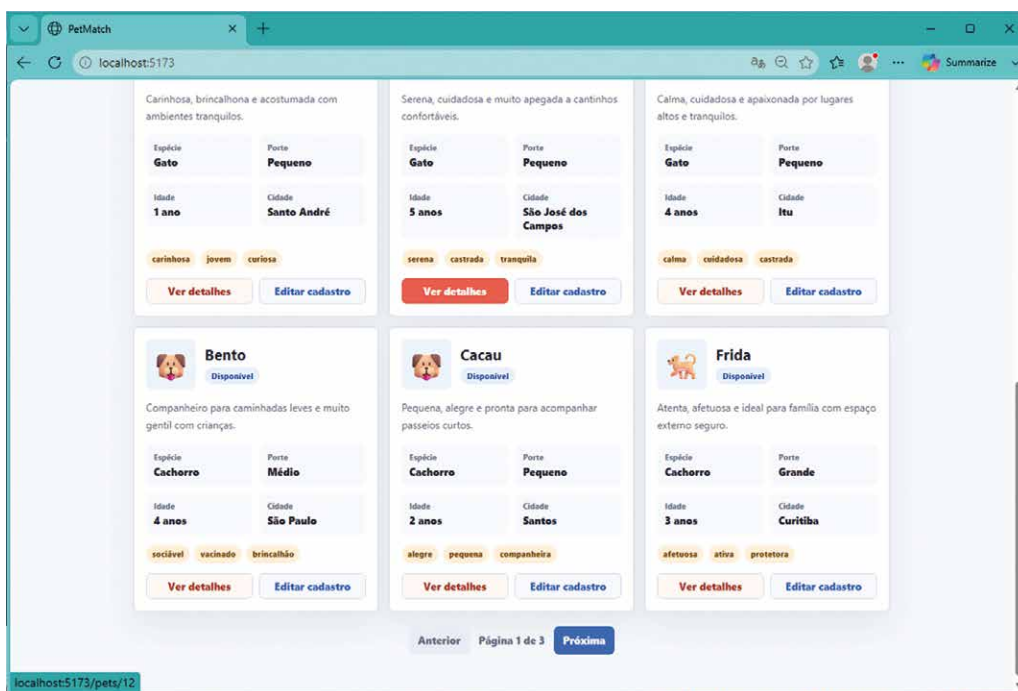


Figura 39 – Página home com React e a inclusão do botão “Ver detalhes” nos cards de cada pet

O card de pet também passa a participar da navegação React. No projeto `PetMatch.Frontend`, o arquivo `src/components/PetCard.tsx` adiciona a ação `Ver detalhes`, apontando para `/pets/{id}` (figura 39). A ação de edição pode continuar apontando para a rota ASP.NET de gerenciamento.

```

import { Link } from 'react-router-dom';
import { PetResponse } from '../api/petmatchClient';

type PetCardProps = {
  pet: PetResponse;
};

export function PetCard({ pet }: PetCardProps) {
  return (
    <article className="pet-card">

```



```

<div className="pet-icon">{pet.icone}</div>
<div className="pet-card-content">
  <h2>{pet.nome}</h2>
  <p>{pet.descricao}</p>

  <div className="pet-meta">
    <span>{pet.especie}</span>
    <span>{pet.porte}</span>
    <span>{pet.idadeAproximada} ano(s)</span>
    <span>{pet.cidade}</span>
  </div>

  <div className="card-actions">
    <Link className="card-action" to={`\/pets/${pet.id}`}>Ver detalhes</Link>
    <a className="card-action secondary" href={`\/Pets/Edit/${pet.id}`}>
Editar cadastro</a>
  </div>
</div>
</article>
);
}

```

Esse componente conecta a lista à página de detalhe. `Link` realiza navegação controlada pelo React Router, mantendo a experiência dentro do front-end. O link de edição continua usando a página ASP.NET, encaminhada pelo proxy de desenvolvimento quando necessário.

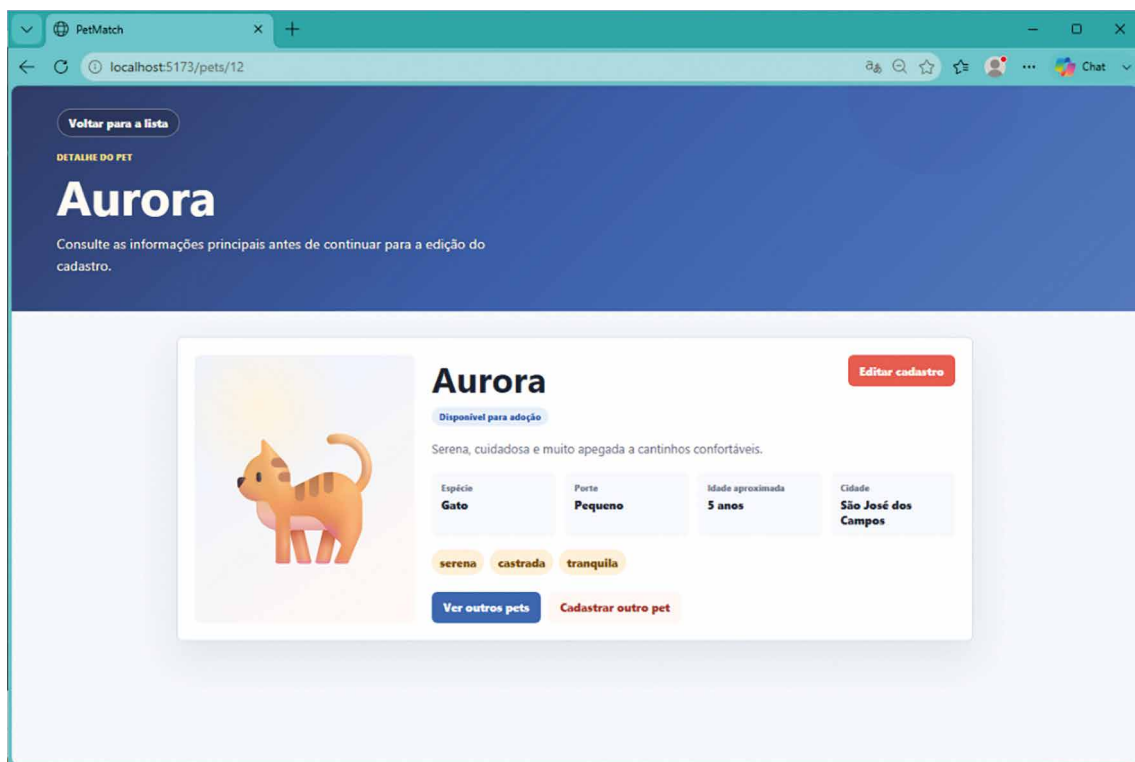


Figura 40 – Página /pets/12 que exibe os detalhes da gatinha Aurora

A página de detalhe fica em `src/pages/PetDetailsPage.tsx`, no projeto `PetMatch.Frontend`. Ela lê o parâmetro `id` da URL, chama `obterPet`, exibe carregamento, erro ou sucesso, e apresenta o pet em uma composição visual mais rica (figura 40).

```
import { Link, useParams } from 'react-router-dom';
import { useEffect, useState } from 'react';
import { obterPet, PetResponse } from '../api/petmatchClient';
import { RequestStatus } from '../components/RequestStatus';

export function PetDetailsPage() {
  const { id } = useParams();
  const [pet, setPet] = useState<PetResponse | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    const petId = Number(id);

    if (!Number.isInteger(petId) || petId <= 0) {
      setError('O endereço informado não corresponde a um pet válido.');
```

setLoading(false);

return;

}

obterPet(petId)

.then(setPet)

.catch(() => setError('Não encontramos este pet no momento.'))

.finally(() => setLoading(false));

}, [id]);

if (loading) {

return <RequestStatus title="Carregando detalhes" description="Estamos buscando as informações deste pet." />;

}

if (error || pet === null) {

return <RequestStatus tone="error" title="Pet não encontrado"

description={error} />;

}

return (

<main className="details-shell">

<Link to="/" className="back-link">Voltar para a lista</Link>

<section className="details-card">

<div className="details-icon">{pet.icone}</div>

<div>

<p className="eyebrow">{pet.especie} para adoção</p>

<h1>{pet.nome}</h1>

<p>{pet.descricao}</p>

```

<div className="pet-meta">
  <span>{pet.porte}</span>
  <span>{pet.idadeAproximada} ano(s)</span>
  <span>{pet.cidade}</span>
</div>

<div className="pet-tags">
  {pet.caracteristicas.map(caracteristica => (
    <span key={caracteristica}>{caracteristica}</span>
  ))}
</div>

<div className="details-actions">
  <a className="primary-action" href={`\/Pets/Edit/${pet.id}`}>Editar
cadastro</a>
  <a className="secondary-action" href="\/Pets/Create">Cadastrar outro
pet</a>
</div>
</div>
</section>
</main>
);
}

```

Essa página mostra como o roteamento e o consumo HTTP trabalham juntos. A URL define qual pet deve ser buscado, `useParams` fornece o identificador, `obterPet` chama a API e o estado da página decide o que será exibido. Um identificador inexistente, como `/pets/99999`, produz uma mensagem amigável (figura 41), pois a API retorna `Problem Details` e o cliente transforma a falha em estado visual.

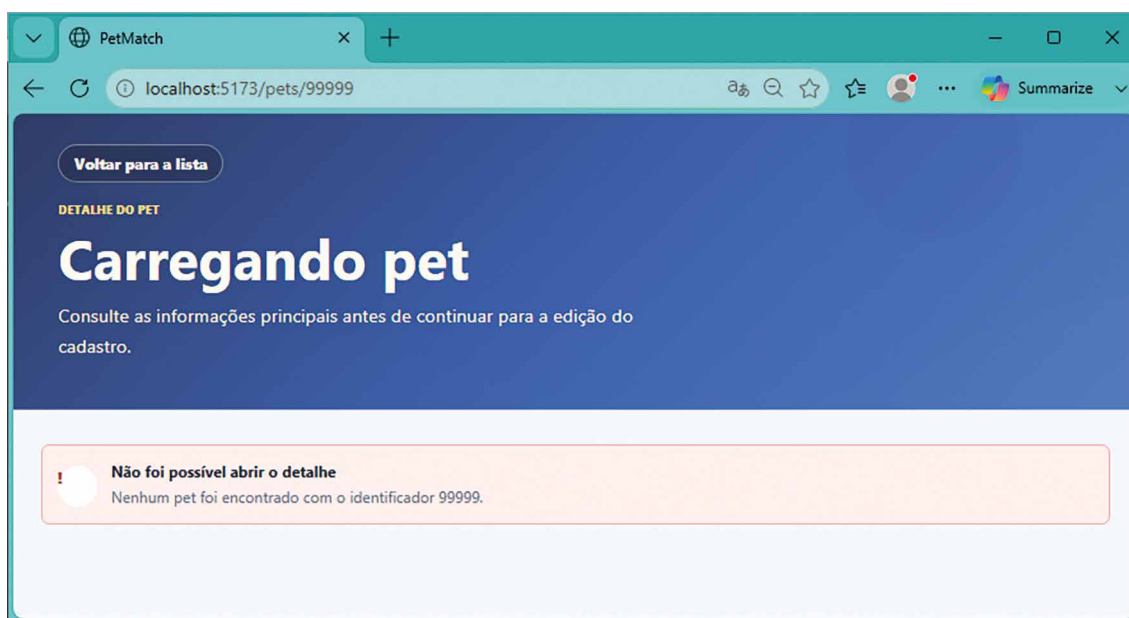


Figura 41 – Mensagem amigável para um identificador de pet inexistente na base de dados `http://localhost:5173/pets/99999`

Os componentes `RequestStatus.tsx` e `EmptyState.tsx`, no projeto `PetMatch.Frontend`, padronizam mensagens de carregamento, erro e ausência de resultados. O objetivo é evitar mensagens soltas e manter a experiência consistente entre lista e detalhe. O arquivo `src/styles/petmatch.css` foi ampliado com estilos para página de detalhe, estados visuais, navegação e ações. O resultado esperado no navegador é uma interface React com lista paginada, filtros funcionando, cards com Ver detalhes, página `/pets/{id}` com ícone, status, descrição, fatos, características e ações para voltar, editar ou cadastrar outro pet.

6 AUTENTICAÇÃO E AUTORIZAÇÃO PONTA A PONTA

Em aplicações web, nem todas as interações podem ser tratadas como acessos anônimos e indiferenciados. À medida que um sistema passa a receber dados pessoais, registrar ações, proteger funcionalidades administrativas e oferecer experiências específicas para diferentes perfis de usuário, torna-se necessário estabelecer mecanismos confiáveis de identidade e controle de acesso. Autenticação e autorização formam a base dessa proteção. A autenticação responde à pergunta sobre quem está utilizando a aplicação; a autorização responde à pergunta sobre o que essa pessoa pode fazer depois de identificada. Embora sejam conceitos relacionados, eles exercem funções distintas e complementares na segurança e na coerência do sistema.

A autenticação representa o processo pelo qual a aplicação verifica a identidade de um usuário. Em uma aplicação web, isso normalmente envolve credenciais, provedores de identidade, emissão de sessão, cookies, tokens ou outros mecanismos que permitam reconhecer uma pessoa ou um sistema em requisições posteriores. O ponto essencial é que o servidor precisa receber alguma evidência confiável de identidade para associar ações a um sujeito determinado. Sem autenticação, todas as solicitações são indistintas; com autenticação, a aplicação passa a saber que determinada requisição foi feita por alguém reconhecido, dentro de um contexto controlado.

A autorização, por sua vez, depende da identidade, mas não se confunde com ela. Saber quem é o usuário não significa permitir automaticamente todas as operações. Um adotante pode consultar pets disponíveis e registrar interesse em adoção, mas não deve necessariamente alterar cadastros institucionais. Uma organização responsável pode administrar informações sobre animais sob seus cuidados, mas talvez não tenha permissão para modificar dados de outra instituição. Um administrador pode exercer funções mais amplas, desde que essas permissões estejam claramente definidas. A autorização transforma a identidade em decisões de acesso, aplicando regras sobre recursos, ações e perfis.

No PetMatch, esses conceitos são fundamentais para preservar a responsabilidade do processo de doação e adoção. Um sistema voltado à aproximação entre pessoas e animais precisa proteger informações de contato, controlar quem pode cadastrar pets, definir quem pode alterar a disponibilidade de um animal e garantir que manifestações de interesse sejam associadas a usuários identificáveis. A segurança, nesse contexto, contribui para a confiança entre participantes, para a integridade dos registros e para a rastreabilidade de ações relevantes. Uma aplicação que permite alterações sensíveis sem identidade ou controle de acesso compromete tanto a experiência quanto a finalidade social do sistema.

A ideia de sessão também ocupa lugar importante nessa arquitetura. Em aplicações orientadas a páginas, a sessão pode ser percebida pelo usuário como a continuidade de sua navegação após o login. Ele se identifica uma vez e passa a acessar áreas compatíveis com seu perfil sem repetir credenciais a cada clique. Tecnicamente, essa continuidade precisa ser mantida por mecanismos seguros que permitam ao servidor reconhecer requisições subsequentes. Em aplicações que expõem APIs consumidas por interfaces cliente, a continuidade pode envolver tokens de acesso, renovação de credenciais, armazenamento cuidadoso no cliente e envio de informações de autenticação em cada chamada protegida. O objetivo, em ambos os casos, é manter a identidade de forma coerente sem abrir espaço para falsificação, vazamento ou uso indevido.

Quando a aplicação possui uma API e uma interface cliente separadas, a segurança precisa ser pensada de ponta a ponta. Não basta proteger visualmente um botão ou esconder uma opção na tela. A interface pode melhorar a experiência ao mostrar apenas ações permitidas ao usuário autenticado, mas a API deve continuar sendo a responsável por validar a identidade e aplicar a autorização de modo definitivo. Qualquer endpoint protegido precisa verificar se a requisição traz credenciais válidas e se o usuário possui permissão para executar aquela operação. Essa separação é indispensável porque APIs podem ser chamadas por clientes diferentes, ferramentas de teste, scripts ou aplicações externas. A proteção real deve estar no serviço que executa a ação, não apenas na camada visual.

A coerência entre API e interface cliente é um dos desafios mais importantes da autenticação e da autorização ponta a ponta. O front-end precisa compreender o estado de autenticação, apresentar corretamente telas de login e saída, adaptar a navegação conforme o perfil do usuário e lidar com respostas de acesso negado ou sessão expirada. A API, ao mesmo tempo, precisa expressar suas decisões por meio de respostas HTTP adequadas, distinguindo falta de autenticação, ausência de permissão e erros de validação. Quando essa comunicação é bem definida, o usuário recebe mensagens claras e a aplicação mantém um comportamento previsível. Quando é mal definida, surgem experiências confusas, como páginas que parecem permitir ações posteriormente recusadas ou erros técnicos apresentados sem contexto.

Em muitos casos, a autorização não depende apenas de um papel geral, mas da relação entre usuário e recurso. Uma organização pode alterar os dados de pets sob sua responsabilidade, mas não de todos os pets cadastrados no sistema. Um adotante pode visualizar suas próprias manifestações de interesse, mas não as de outros usuários. Um recurso, portanto, não é protegido apenas por sua categoria, mas pelo vínculo que mantém com a identidade autenticada. Essa forma de autorização aproxima a segurança das regras reais do negócio e impede que permissões excessivamente amplas exponham dados ou operações indevidas.

A autenticação informa quem está realizando a ação; a autorização determina se a ação é permitida; e a validação verifica se os dados enviados respeitam as regras esperadas. Esses mecanismos não substituem uns aos outros. Um usuário autenticado e autorizado ainda pode enviar dados inválidos. Um dado formalmente válido ainda pode ser proibido para determinado perfil. Uma requisição sem identidade pode nem chegar à etapa de processamento do domínio. A arquitetura segura depende da

combinação desses controles, distribuídos de forma adequada ao longo do pipeline, dos endpoints, dos serviços de aplicação e da interface.

No PetMatch, a adoção responsável exige que o sistema trate confiança como requisito arquitetural. Usuários devem compreender quando estão navegando livremente, quando precisam se identificar e por que determinada funcionalidade exige permissão específica. Organizações precisam ter segurança para cadastrar informações reais sobre animais. Adotantes precisam confiar que seus dados não serão expostos indevidamente. Administradores precisam de recursos de gestão sem que isso signifique fragilizar as fronteiras de acesso. A autenticação e a autorização tornam possível organizar essas relações de maneira técnica e conceitualmente sólida.

Este tópico introduz autenticação e autorização como mecanismos de identidade, sessão, proteção de recursos e coerência entre API e interface cliente. A expressão ponta a ponta indica que a segurança não deve ser concentrada em um único ponto visível da aplicação, mas distribuída de modo coordenado desde a experiência do usuário até os endpoints protegidos, desde o estado mantido no cliente até as decisões aplicadas no servidor. Compreender essa arquitetura é essencial para construir aplicações ASP.NET Core capazes de lidar com usuários reais, dados sensíveis e operações diferenciadas sem comprometer clareza, usabilidade e responsabilidade técnica.

6.1 ASP.NET Identity, cookies × JWT e roles/claims

A segurança em aplicações web começa pela distinção entre identidade, autenticação e autorização. Identidade é a representação de uma pessoa ou entidade dentro do sistema, normalmente associada a informações como identificador, e-mail, nome e dados de perfil. Autenticação é o processo pelo qual a aplicação confirma que o usuário é quem afirma ser, em geral por meio de credenciais, login externo, certificado ou outro mecanismo reconhecido. Autorização é a decisão tomada depois da autenticação: a aplicação verifica se aquele usuário pode executar determinada ação, abrir uma página ou acessar um recurso. Um sistema bem projetado trata essas três ideias como partes relacionadas, mas separadas.

O ASP.NET Identity é a infraestrutura do ASP.NET Core para lidar com usuários, senhas, roles, claims, tokens internos, cookies de autenticação e armazenamento desses dados com Entity Framework Core. Ele fornece serviços para criar usuários, validar senhas, entrar e sair da aplicação, associar usuários a papéis e consultar informações de identidade durante a execução de uma requisição. Ao ser integrado ao EF Core, Identity cria tabelas próprias no banco para usuários, roles, claims, logins e relacionamentos, permitindo que a autenticação participe do mesmo ambiente persistente da aplicação.

Quadro 15

Conceito	O que é no ASP.NET Core Identity	Para que serve	Exemplo prático
Usuário	A entidade que representa uma pessoa ou conta no sistema, geralmente baseada em <code>IdentityUser</code>	Guardar dados como Id, <code>UserName</code> , Email, telefone, confirmação de e-mail e outros campos de perfil	Um registro na tabela <code>AspNetUsers</code> para <code>ana@email.com</code>
Senha	Credencial secreta do usuário, armazenada como hash, não como texto puro	Permitir autenticação local com e-mail/usuário e senha	O usuário digita a senha; o Identity compara o hash calculado com o hash salvo
Role	Um papel, grupo ou função atribuída ao usuário, como Admin, Manager ou User	Facilitar autorização baseada em categorias amplas de acesso	<code>[Authorize(Roles = "Admin")]</code> restringe uma página a administradores
Claim	Uma declaração sobre o usuário: um par tipo/valor, como <code>Department = Finance</code> ou <code>Permission = CanEditInvoices</code>	Representar informações ou permissões mais granulares que roles	Um usuário pode ter a claim <code>Permission:EditProducts</code>
Login	O processo ou vínculo usado para autenticar o usuário. Pode ser login local ou externo	Associar uma conta Identity a credenciais locais ou provedores externos	Login com senha local, Google, Microsoft, GitHub etc.
Tokens internos	Tokens gerados pelo Identity para operações específicas, como confirmação de e-mail, redefinição de senha e autenticação de dois fatores	Executar ações sensíveis sem expor senha ou estado diretamente	Link de "redefinir senha" contém um token temporário validado pelo Identity
Cookies de autenticação	Cookie emitido após login bem-sucedido, contendo uma identidade autenticada, normalmente com claims	Manter o usuário logado entre requisições HTTP	Depois do login, o navegador envia o cookie e o ASP.NET Core reconhece o usuário

A autenticação por cookie é uma forma tradicional e eficiente de manter uma sessão web no navegador. Depois que o usuário entra com sucesso, o servidor emite um cookie protegido. Em requisições seguintes, o navegador envia esse cookie automaticamente para o mesmo site e o ASP.NET Core reconstrói a identidade do usuário a partir dele. Esse modelo se ajusta bem a aplicações web que possuem páginas MVC, formulários e navegação pelo navegador, como ocorre nas telas protegidas do PetMatch.

Para entender melhor, pense que quando o usuário faz login, o servidor responde ao navegador com um cookie:

```
Set-Cookie:.AspNetCore.Identity.Application=CfDJ8HqL8X9aQ3...mZp2w
```

Depois disso, em cada nova requisição para o mesmo site, o navegador envia automaticamente:

```
Cookie:.AspNetCore.Identity.Application=CfDJ8HqL8X9aQ3...mZp2w
```

A ideia por trás desse envio é: "Servidor, aqui está o cookie que você me deu antes. Veja se eu ainda estou logado". Então, o ASP.NET Core lê esse cookie e descobre internamente:

```
Usuário: Ana
```

```
Id: 42
```

Role: Tutor

Está autenticada: Sim

Já o JWT, ou JSON Web Token, representa outro modelo de transporte de identidade. Um JWT é um token textual, geralmente enviado no cabeçalho Authorization, muito comum em APIs consumidas por clientes independentes, aplicações móveis, integrações entre serviços e front-ends distribuídos separadamente. Ele carrega declarações assinadas sobre o usuário e precisa ser validado a cada requisição.

Para visualizar melhor, pense que quando o usuário faz login em uma API, o servidor responde com um token JWT, geralmente no corpo da resposta:

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiJ9.  
eyJzdWIiOiI0MiIsIm5hbWUiOiJBbmEiLCJyb2xlIjoiaVHV0b3IifQ.assinatura"  
}
```

Depois disso, o cliente guarda esse token e o envia manualmente nas próximas requisições, normalmente no cabeçalho Authorization:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.  
eyJzdWIiOiI0MiIsIm5hbWUiOiJBbmEiLCJyb2xlIjoiaVHV0b3IifQ.assinatura
```

Cookies e JWT podem participar de arquiteturas seguras, mas respondem a contextos diferentes. Neste subtópico, o PetMatch usa cookies com ASP.NET Identity, porque o fluxo prático envolve navegador, páginas MVC protegidas e integração com a experiência React por meio do mesmo back-end.



Saiba mais

Uma referência fundamental sobre cookies HTTP, definindo os cabeçalhos Cookie e Set-Cookie e explicando seu papel na manutenção de estado em um protocolo originalmente sem estado, é:

BARTH, A. *HTTP State Management Mechanism*. IETF, 2011. Disponível em: <https://tinyurl.com/7yu7yrb>. Acesso em: 8 jul. 2026.

Já a especificação clássica do JWT, útil para compreender claims, assinatura, representação compacta de tokens e uso em autenticação e autorização em APIs, é:

JONES, M. B.; BRADLEY, J.; SAKIMURA, N. *JSON Web Token (JWT)*. IETF, 2015. RFC 7519. Disponível em: <https://tinyurl.com/3zfbe9uw>. Acesso em: 8 jul. 2026.

Roles e claims expressam autorização em níveis diferentes. Uma role representa um papel amplo do usuário no sistema, como Voluntario ou Administrador. Ela é adequada quando uma regra pode ser descrita por pertencimento a um grupo. Uma claim representa uma afirmação sobre o usuário, como nome, cidade, identificador, permissão específica ou vínculo institucional. Claims permitem decisões mais granulares e também alimentam telas de perfil. No ASP.NET Core, tanto roles quanto claims passam a compor o usuário autenticado e podem ser consultadas por controllers, views e políticas de autorização.

O pipeline HTTP precisa conhecer autenticação e autorização para aplicar essas decisões durante a requisição. A autenticação identifica o usuário associado à requisição atual. A autorização usa essa identidade para permitir ou negar acesso. A ordem dos middlewares é significativa: primeiro, a aplicação precisa autenticar, depois autorizar. Quando essa ordem é respeitada, atributos como `[Authorize]` conseguem proteger controllers e actions de forma declarativa.

No PetMatch, a vitrine React continua pública. O usuário pode abrir a home, filtrar pets, navegar pela lista e acessar detalhes (figura 42) sem criar conta. As ações de cadastro e edição, por outro lado, passam a exigir autenticação e role adequada, pois alteram informações do sistema. Ao tentar cadastrar ou editar sem entrar, o usuário é levado à tela de login (figura 43). Ao entrar ou criar conta (figura 44), passa a acessar perfil (figura 45) e ações protegidas (figura 46) conforme sua role. Esse comportamento torna a segurança parte natural do fluxo de adoção responsável, sem transformar a aplicação em uma tela técnica.

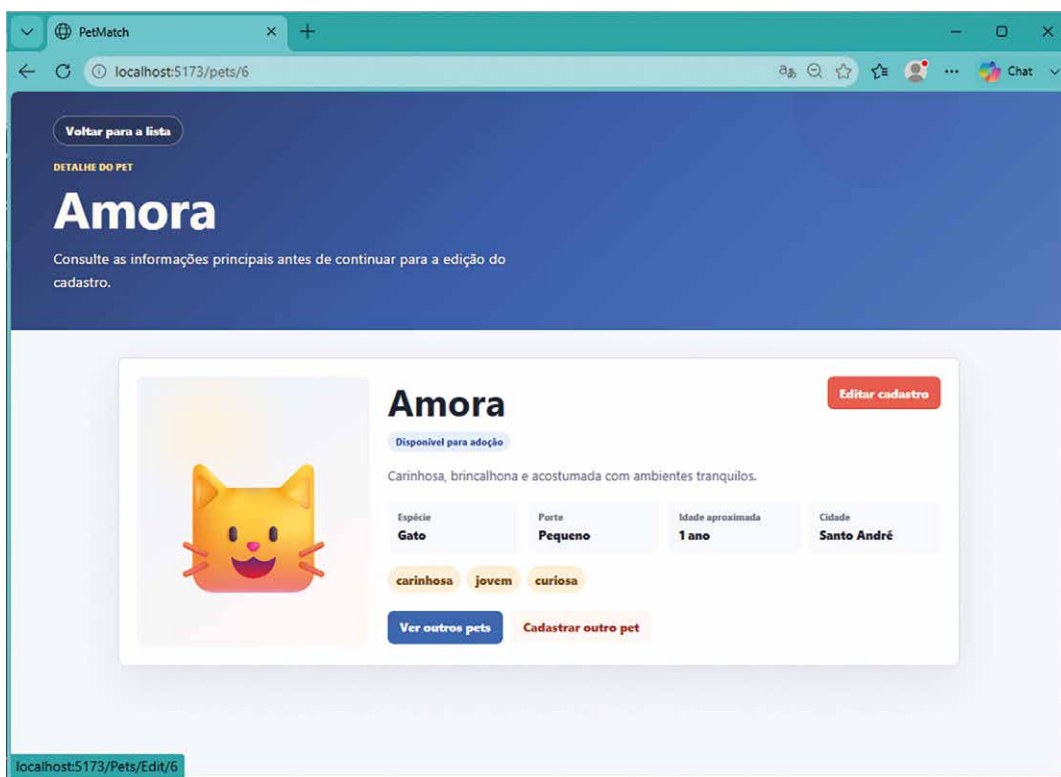


Figura 42 – Tela React que exibe os detalhes do pet sem necessidade de login.
O botão “Editar cadastro” irá exigir (figura 43)

Figura 43 – Tela de login do ASP.NET após a tentativa de edição cadastral da gatinha Amora

Figura 44 – Tela para criação de nova conta. Note que há validação do campo de senha com política de senha forte

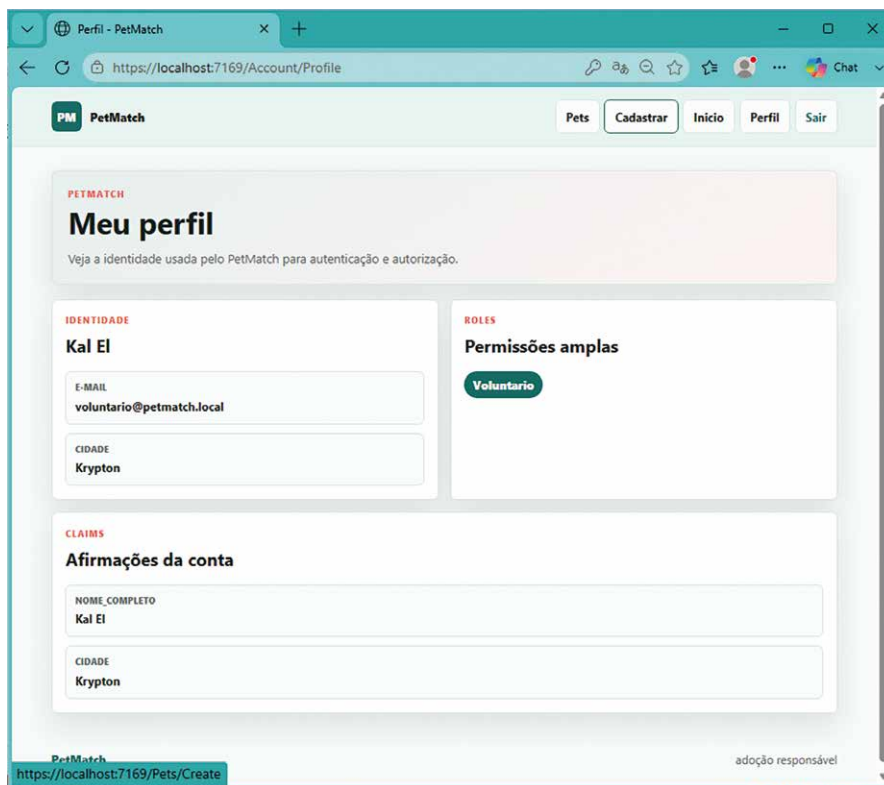


Figura 45 – Conta de usuário criada conforme definições da figura 44

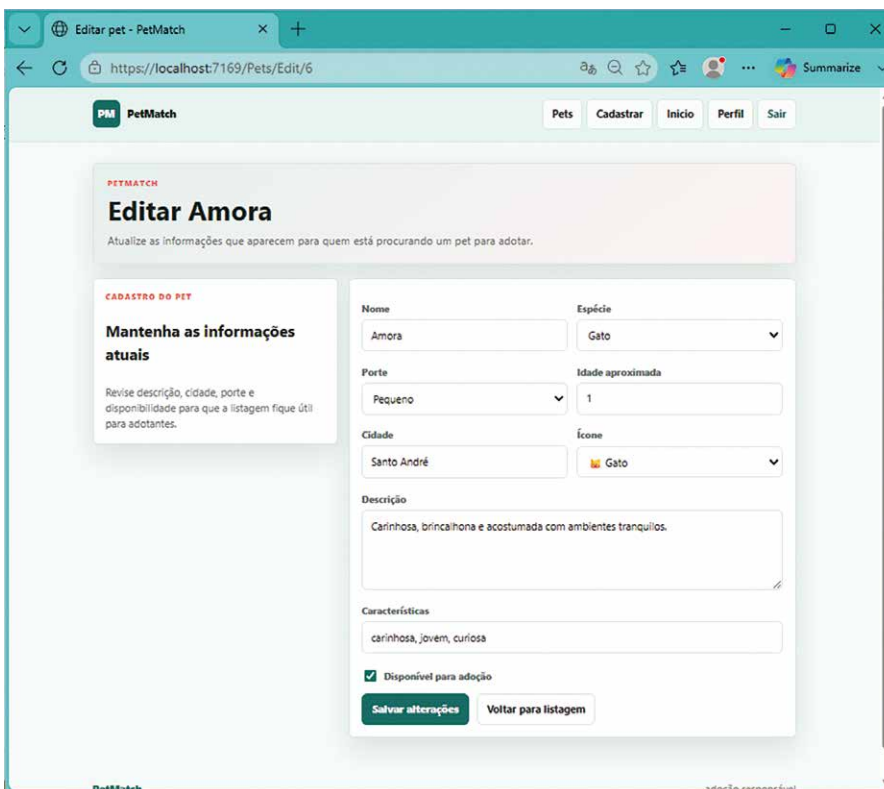


Figura 46 – Edição de cadastro da gatinha Amora liberada para o novo usuário cadastrado (figura 45)

A estrutura implementada fica no projeto PetMatch.Web. Foram criados ApplicationUser.cs e PetMatchRoles.cs em Models, ViewModels de conta em ViewModels, AccountController.cs em Controllers, views em Views/Account, além de ajustes em PetMatchDbContext.cs, PetMatchSeed.cs, PetsController.cs, _Layout.cshtml, petmatch-mvc.css, Program.cs e na migration 202607050001_AddIdentitySchema. O projeto PetMatch.Frontend foi preservado. O pacote NuGet adicionado foi Microsoft.AspNetCore.Identity.EntityFrameworkCore 10.0.9.

No projeto PetMatch.Web, o arquivo Models/ApplicationUser.cs define o usuário da aplicação. Ele deriva de IdentityUser, recebendo toda a infraestrutura de Identity e acrescentando dados próprios do PetMatch.

```
using Microsoft.AspNetCore.Identity;

namespace PetMatch.Web.Models;

public sealed class ApplicationUser : IdentityUser
{
    public string NomeCompleto { get; set; } = string.Empty;

    public string Cidade { get; set; } = string.Empty;
}
```

Esse modelo permite que o usuário tenha e-mail, senha protegida e identificador fornecidos pelo Identity, além de informações de perfil usadas pela aplicação. NomeCompleto e Cidade tornam a tela de perfil mais concreta e mostram que Identity pode ser estendido com propriedades do domínio. As roles usadas pela aplicação ficam centralizadas em Models/PetMatchRoles.cs, no projeto PetMatch.Web. Isso evita espalhar textos literais por controllers e seeds.

```
namespace PetMatch.Web.Models;

public static class PetMatchRoles
{
    public const string Administrador = "Administrador";

    public const string Voluntario = "Voluntario";
}
```

Esse arquivo dá nome às permissões amplas do sistema. Voluntario representa o usuário autorizado a cadastrar e editar pets. Administrador representa um papel superior, capaz de acessar os mesmos fluxos protegidos.

Para armazenar usuários e roles no mesmo banco SQLite, o contexto passa a herdar de IdentityDbContext<ApplicationUser>. No projeto PetMatch.Web, o arquivo Data/PetMatchDbContext.cs preserva o DbSet<Pet> e incorpora as tabelas do Identity.

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using PetMatch.Web.Models;
```

```
namespace PetMatch.Web.Data;

public sealed class PetMatchDbContext : IdentityDbContext<ApplicationUser>
{
    public PetMatchDbContext(DbContextOptions<PetMatchDbContext> options)
        : base(options)
    {
    }

    public DbSet<Pet> Pets => Set<Pet>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Pet>(entity =>
        {
            entity.HasKey(pet => pet.Id);
            entity.Property(pet => pet.Nome).HasMaxLength(80).IsRequired();
            entity.Property(pet => pet.Especie).HasMaxLength(40).IsRequired();
            entity.Property(pet => pet.Porte).HasMaxLength(30).IsRequired();
            entity.Property(pet => pet.Cidade).HasMaxLength(80).IsRequired();
            entity.Property(pet => pet.RowVersion).IsConcurrencyToken().
IsRequired();
        });
    }
}
```

A chamada `base.OnModelCreating(modelBuilder)` é essencial porque permite que o Identity configure suas próprias tabelas. Depois disso, o mapeamento de `Pet` continua existindo no mesmo contexto. A aplicação passa a ter persistência de domínio e persistência de identidade na mesma unidade de acesso ao banco.

O registro dos serviços de autenticação fica em `Program.cs`, no projeto `PetMatch.Web`. Esse trecho configura Identity, banco, roles, cookie e caminhos das páginas de conta.

```
builder.Services
    .AddIdentity<ApplicationUser, IdentityRole>(options =>
    {
        options.User.RequireUniqueEmail = true;
        options.SignIn.RequireConfirmedAccount = false;
    })
    .AddEntityFrameworkStores<PetMatchDbContext>()
    .AddDefaultTokenProviders();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.Cookie.Name = "PetMatch.Auth";
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
});
```

Esse código conecta `ApplicationUser` e `IdentityRole` ao `PetMatchDbContext`. A configuração do cookie define o nome do cookie de autenticação e os caminhos usados quando uma página protegida exige login ou quando o usuário autenticado não possui permissão suficiente.

No mesmo `Program.cs`, a ordem do pipeline precisa autenticar antes de autorizar. O trecho a seguir aparece depois dos middlewares de arquivos estáticos e antes do mapeamento de rotas e endpoints.

```
app.UseAuthentication();  
app.UseAuthorization();
```

Essas chamadas fazem com que o ASP.NET Core leia o cookie de autenticação, reconstrua o usuário atual e aplique regras de autorização nos controllers protegidos. Sem essa etapa do pipeline, os atributos `[Authorize]` não teriam o contexto necessário para decidir acesso.

O seed da aplicação foi ampliado para preparar roles, sem gravar uma senha fixa no código. No projeto `PetMatch.Web`, o arquivo `Data/PetMatchSeed.cs` cria `Administrador` e `Voluntario`, preservando também o seed dos pets já existentes. A criação de conta ocorre pela tela de cadastro e o novo usuário recebe a role `Voluntario` (figura 45).

```
public static async Task SeedRolesAsync(RoleManager<IdentityRole> roleManager)  
{  
    var roles = new[]  
    {  
        PetMatchRoles.Administrador,  
        PetMatchRoles.Voluntario  
    };  
  
    foreach (var role in roles)  
    {  
        if (!await roleManager.RoleExistsAsync(role))  
        {  
            await roleManager.CreateAsync(new IdentityRole(role));  
        }  
    }  
}
```

Esse código prepara as roles necessárias para autorização. A aplicação não depende de credenciais fixas versionadas no projeto; o usuário cria a própria conta pela interface e recebe a role prevista para participar dos fluxos protegidos do `PetMatch`.

O controller de conta concentra login, cadastro, perfil, logout e acesso negado. No projeto `PetMatch.Web`, o arquivo `Controllers/AccountController.cs` usa `userManager<ApplicationUser>` para criar e consultar usuários, `signInManager<ApplicationUser>` para entrar e sair, e `RoleManager<IdentityRole>` ou operações equivalentes para associar roles.

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (!ModelState.IsValid)
    {
        return View(model);
    }

    var user = new ApplicationUser
    {
        UserName = model.Email,
        Email = model.Email,
        NomeCompleto = model.NomeCompleto,
        Cidade = model.Cidade
    };

    var result = await _userManager.CreateAsync(user, model.Senha);

    if (!result.Succeeded)
    {
        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }

        return View(model);
    }

    await _userManager.AddToRoleAsync(user, PetMatchRoles.Voluntario);
    await _signInManager.SignInAsync(user, isPersistent: false);

    return RedirectToAction(nameof(Profile));
}
```

Esse fluxo mostra a integração entre formulário, ViewModel, Identity e role. O cadastro valida os dados recebidos, cria o usuário com senha protegida pelo Identity, associa a role `Voluntario` e entra automaticamente com o novo usuário. O resultado visual é direto: após o cadastro, o usuário acessa seu perfil.

A proteção das páginas de cadastro e edição de pets fica declarada no controller responsável pelos pets. No projeto `PetMatch.Web`, o arquivo `Controllers/PetsController.cs` protege as actions sensíveis com `[Authorize]`.

```
[Authorize(Roles = $"{PetMatchRoles.Voluntario},{PetMatchRoles.Administrador}")]
[HttpGet]
public IActionResult Create()
{
    return View(new PetCreateViewModel());
}

[Authorize(Roles = $"{PetMatchRoles.Voluntario},{PetMatchRoles.Administrador}")]
[HttpGet]
public async Task<IActionResult> Edit(int id)
{
    // Carrega o pet e exibe o formulário de edição.
}
```

Esse código expressa autorização de forma declarativa. A listagem e o detalhe continuam públicos, enquanto criação e edição exigem usuário autenticado com role adequada. Quando o usuário anônimo tenta abrir uma dessas páginas, o cookie ausente leva ao redirecionamento para login.

A tela de login fica em `Views/Account/Login.cshtml`, no projeto `PetMatch.Web`. Ela usa `Tag Helpers` e o `LoginViewModel` para capturar e-mail, senha e retorno visual de validação.

```
@model LoginViewModel

@{
    ViewData["Title"] = "Entrar no PetMatch";
}

<section class="account-panel">
    <h1>Entrar no PetMatch</h1>
    <p>Acesse sua conta para cadastrar e gerenciar pets disponíveis para adoção.</p>

    <form asp-action="Login" method="post" class="account-form">
        <div asp-validation-summary="ModelOnly" class="validation-summary"></div>

        <label asp-for="Email"></label>
        <input asp-for="Email" autocomplete="email" />
        <span asp-validation-for="Email" class="field-validation"></span>

        <label asp-for="Senha"></label>
        <input asp-for="Senha" type="password" autocomplete="current-password" />
        <span asp-validation-for="Senha" class="field-validation"></span>

        <button type="submit">Entrar</button>
    </form>

    <a asp-controller="Account" asp-action="Register">Criar conta</a>
</section>
```

Essa view transforma autenticação em uma experiência visual coerente com o PetMatch. O usuário não vê uma tela técnica; ele encontra uma página de entrada com texto claro, validação e caminho para cadastro.

A página de perfil usa `ProfileViewModel` e exibe dados do usuário autenticado. Ela também pode mostrar roles e claims principais de forma compreensível, ajudando o leitor a perceber como informações de identidade aparecem na aplicação.

```
@model ProfileViewModel

@{
    ViewData["Title"] = "Meu perfil";
}
```



```
<section class="account-panel">
  <h1>Meu perfil</h1>

  <dl class="profile-list">
    <dt>Nome</dt>
    <dd>@Model.NomeCompleto</dd>

    <dt>E-mail</dt>
    <dd>@Model.Email</dd>

    <dt>Cidade</dt>
    <dd>@Model.Cidade</dd>

    <dt>Perfil de acesso</dt>
    <dd>@Model.Role</dd>
  </dl>

  <form asp-controller="Account" asp-action="Logout" method="post">
    <button type="submit">Sair</button>
  </form>
</section>
```

Essa tela demonstra o uso prático dos dados de identidade depois do login. O perfil dá ao usuário uma referência clara de quem está autenticado e de qual papel está associado à sua conta. O layout compartilhado também foi ajustado. Em `Views/Shared/_Layout.cshtml`, no projeto `PetMatch.Web`, a navegação passa a refletir o estado de autenticação, exibindo entrada e cadastro para visitantes, ou perfil e saída para usuários autenticados. O CSS `wwwroot/css/petmatch-mvc.css` recebeu estilos para formulários de conta, perfil, mensagens e acesso negado. O resultado esperado é uma experiência visual completa: a vitrine React permanece pública; ao clicar em cadastrar ou editar sem login, o usuário vê a tela de entrada; depois de criar conta, acessa o perfil e as páginas protegidas; se não possuir role adequada, vê uma página amigável de acesso negado.

6.2 SPA – login/logout, armazenamento seguro de tokens e rotas protegidas

Uma aplicação SPA precisa tratar autenticação como parte do estado da interface. Ao abrir a aplicação, o navegador ainda não sabe se há uma sessão válida no servidor. A primeira tarefa do front-end é descobrir esse estado: há usuário autenticado, a sessão expirou, a chamada está carregando ou o visitante está anônimo. Essa verificação influencia a navegação, os botões visíveis, as páginas protegidas e as mensagens exibidas ao usuário. Em React, portanto, autenticação não se resume a uma tela de entrada; ela se torna um estado global que precisa ser consultado por diferentes páginas e componentes.

Login é o fluxo em que o usuário envia credenciais ao servidor e recebe uma sessão válida quando as credenciais são aceitas. Logout encerra essa sessão e faz a interface voltar ao estado anônimo. Em uma SPA, esses dois fluxos também precisam atualizar o estado local da aplicação. Quando o login termina, a barra de navegação passa a mostrar perfil e saída; quando o logout termina, a interface volta a mostrar entrada e criação de conta. Esse ajuste visual é tão importante quanto a chamada HTTP, pois o usuário precisa perceber claramente sua condição dentro do sistema.

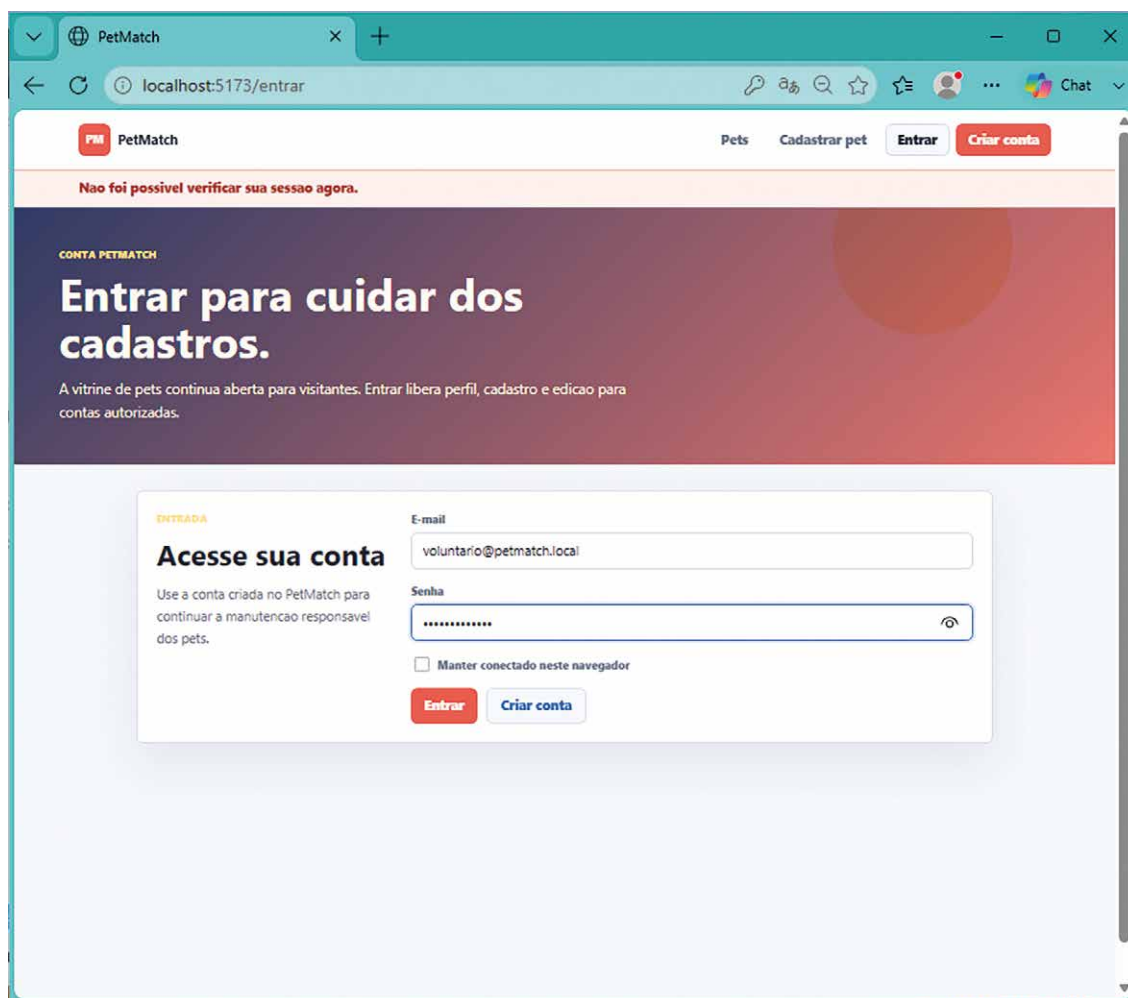


Figura 47 – Tela de login do PetMatch. Note que a sessão ainda não foi verificada

O armazenamento seguro de tokens é um dos pontos mais sensíveis em aplicações no navegador. Quando uma aplicação usa JWT explícito, o front-end recebe um token e precisa decidir como mantê-lo entre requisições. Armazenamentos acessíveis por JavaScript, como `localStorage` e `sessionStorage`, ampliam a exposição do token se houver execução indevida de scripts na página. Por isso, aplicações web que usam o navegador como cliente principal frequentemente preferem um cookie protegido emitido pelo servidor, com atributos como `HttpOnly`, `Secure` e `SameSite`. Nesse modelo, o JavaScript não lê o token, o navegador envia o cookie nas requisições e o servidor valida a sessão.

JWT continua sendo um formato importante para APIs consumidas por clientes independentes, integrações entre serviços e aplicações móveis. Ele carrega declarações assinadas e costuma ser enviado no cabeçalho `Authorization`. Seu uso exige decisões sobre expiração, renovação, armazenamento e revogação. No fluxo implementado aqui, o PetMatch usa o caminho já preparado no back-end: ASP.NET Identity com cookie de autenticação. A SPA React interage com esse cookie por meio de chamadas HTTP com `credentials: "include"`, mantendo o token fora do alcance direto do código JavaScript.

As rotas protegidas no front-end organizam a experiência do usuário. Quando uma rota como `/perfil` exige autenticação, a SPA pode verificar o estado da sessão antes de renderizar a página. Se a sessão ainda está sendo carregada, mostra uma mensagem temporária; se o usuário está anônimo, redireciona para a tela de entrada; se o usuário está autenticado, exibe o conteúdo solicitado. Essa proteção melhora a navegação e evita telas incoerentes. A proteção de recursos continua no ASP.NET Core, por meio de cookies, Identity, roles e atributos de autorização.

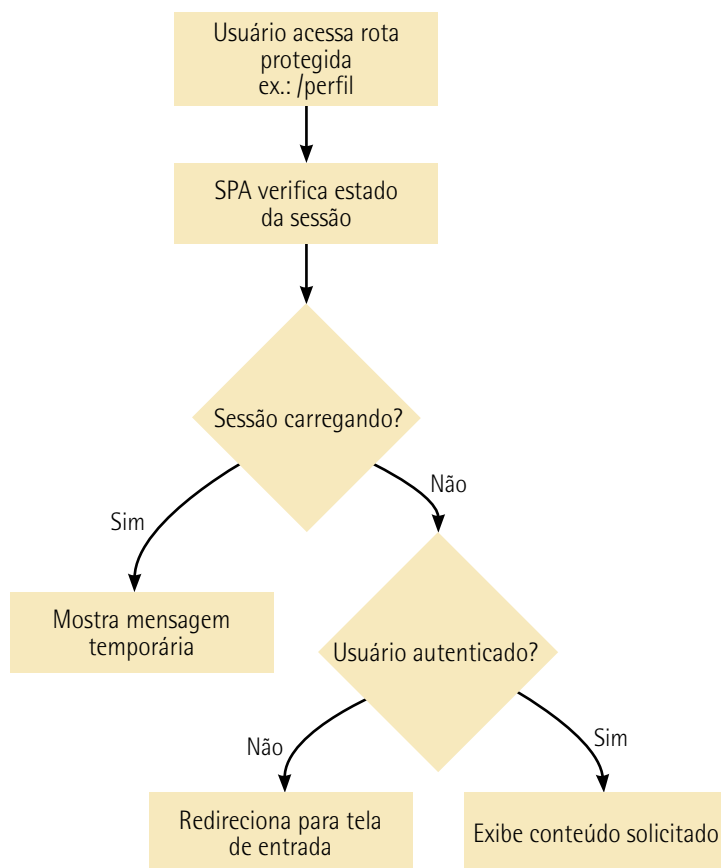


Figura 48 – Fluxo de acesso a rotas protegidas na SPA

O estado de autenticação também é uma requisição. Ao iniciar a SPA, o React chama o servidor para saber quem é o usuário atual. Essa chamada pode retornar usuário autenticado, usuário anônimo, falha de rede ou sessão encerrada. A aplicação precisa transformar cada resposta em uma experiência visual compreensível. A mesma lógica vale para login, cadastro e logout: cada operação possui carregamento, sucesso e erro. Assim, os conceitos de estados de requisição, estudados nas listagens e detalhes de pets, passam a organizar também o fluxo de conta.

No PetMatch, a vitrine e a página de detalhe continuam públicas. Qualquer visitante pode conhecer os pets disponíveis. As ações de cadastro e edição permanecem protegidas no ASP.NET Core, pois alteram dados do sistema. A SPA passa a oferecer telas próprias de entrada, criação de conta e perfil, integradas à paleta visual React. Ao entrar pela SPA, o usuário recebe o cookie `PetMatch.Auth` emitido pelo servidor e consegue acessar páginas protegidas, inclusive páginas ASP.NET MVC, pelo mesmo navegador.

A estrutura implementada envolve os dois projetos. No back-end `PetMatch.Web`, foram criados contratos de autenticação em `Contracts`, os endpoints em `Endpoints/AuthEndpoints.cs` e ajustes em `Program.cs`. No front-end `PetMatch.Frontend`, foram criados `src/api/authClient.ts`, `src/auth/AuthContext.tsx`, `components/AppNav.tsx`, `components/ProtectedRoute.tsx`, `pages/LoginPage.tsx`, `pages/RegisterPage.tsx` e `pages/ProfilePage.tsx`, além de ajustes em `App.tsx`, `main.tsx`, `vite.config.ts` e `styles/petmatch.css`. Nenhum pacote novo foi adicionado; o projeto reaproveita ASP.NET Identity e `react-router-dom`.

No projeto `PetMatch.Web`, os contratos de autenticação definem a forma das requisições e respostas JSON usadas pela SPA. O arquivo `Contracts/AuthLoginRequest.cs` representa a entrada de login; `Contracts/AuthRegisterRequest.cs` representa o cadastro; e `Contracts/AuthUserResponse.cs` representa o estado atual da sessão.

```
namespace PetMatch.Web.Contracts;

public sealed record AuthLoginRequest(
    string Email,
    string Senha,
    string? ReturnUrl);

public sealed record AuthRegisterRequest(
    string NomeCompleto,
    string Cidade,
    string Email,
    string Senha,
    string? ReturnUrl);

public sealed record AuthUserResponse(
    bool IsAuthenticated,
    string? Email,
    string? NomeCompleto,
    IReadOnlyList<string> Roles,
    IReadOnlyList<string> Claims);
```

Esses contratos tornam explícito o diálogo entre React e ASP.NET Core. A SPA envia e-mail e senha para login, envia dados de cadastro para criar conta e recebe um objeto de sessão que informa se há usuário autenticado, quais roles ele possui e quais claims principais estão disponíveis.

Os endpoints de autenticação ficam em `Endpoints/AuthEndpoints.cs`, no projeto `PetMatch.Web`. Eles usam os serviços já configurados do ASP.NET Identity e publicam operações JSON para a SPA.

```
using Microsoft.AspNetCore.Identity;
using PetMatch.Web.Contracts;
using PetMatch.Web.Models;

namespace PetMatch.Web.Endpoints;

public static class AuthEndpoints
{
    public static IEndpointRouteBuilder MapAuthEndpoints(this
        IEndpointRouteBuilder endpoints)
    {
```

```

var group = endpoints.MapGroup("/api/auth");

group.MapGet("/me", async (HttpContext httpContext,
userManager<ApplicationUser> userManager) =>
{
    if (httpContext.User.Identity?.IsAuthenticated != true)
    {
        return Results.Ok(new AuthUserResponse(false, null, null, [],
[]));
    }

    var user = await userManager.GetUserAsync(httpContext.User);
    var roles = user is null ? [] : await userManager.GetRolesAsync(user);
    var claims = httpContext.User.Claims.Select(claim => $"{claim.Type}:
{claim.Value}").ToList();

    return Results.Ok(new AuthUserResponse(
        true,
        user?.Email,
        user?.NomeCompleto,
        roles,
        claims));
});

group.MapPost("/logout", async (SignInManager<ApplicationUser>
signInManager) =>
{
    await signInManager.SignOutAsync();
    return Results.NoContent();
});

return endpoints;
}

```

Esse trecho mostra duas operações centrais. GET /api/auth/me informa à SPA se existe sessão ativa. POST /api/auth/logout encerra a sessão no servidor. As operações de login e cadastro seguem o mesmo grupo e usam SignInManager<ApplicationUser> e UserManager<ApplicationUser> para validar credenciais, criar usuário, atribuir role `Voluntario` e emitir o cookie.

O comportamento do cookie para chamadas de API é ajustado em Program.cs, no projeto PetMatch.Web. Em páginas MVC, o redirecionamento para login é conveniente. Em endpoints /api, a SPA precisa receber status HTTP para decidir o que mostrar.

```

builder.Services.ConfigureApplicationCookie(options =>
{
    options.Cookie.Name = "PetMatch.Auth";
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
});

```

```
options.Events.OnRedirectToLogin = context =>
{
  if (context.Request.Path.StartsWithSegments("/api"))
  {
    context.Response.StatusCode = StatusCodes.Status401Unauthorized;
    return Task.CompletedTask;
  }

  context.Response.Redirect(context.RedirectUri);
  return Task.CompletedTask;
};

options.Events.OnRedirectToAccessDenied = context =>
{
  if (context.Request.Path.StartsWithSegments("/api"))
  {
    context.Response.StatusCode = StatusCodes.Status403Forbidden;
    return Task.CompletedTask;
  }

  context.Response.Redirect(context.RedirectUri);
  return Task.CompletedTask;
};
});
```

Esse código separa o comportamento de páginas e APIs. Uma página protegida pode levar o usuário à tela de login. Uma chamada da SPA recebe 401 ou 403, permitindo que o React mostre uma mensagem ou redirecione para /entrar sem receber HTML inesperado.

No front-end, o cliente de autenticação fica em `src/api/authClient.ts`, no projeto `PetMatch.Frontend`. Ele usa `fetch` com `credentials: "include"` para enviar e receber o cookie de autenticação.

```
export type AuthUser = {
  isAuthenticated: boolean;
  email: string | null;
  nomeCompleto: string | null;
  roles: string[];
  claims: string[];
};

export type LoginRequest = {
  email: string;
  senha: string;
  returnUrl?: string;
};

export async function getCurrentUser(): Promise<AuthUser> {
  const response = await fetch('/api/auth/me', {
    credentials: 'include'
  });

  if (!response.ok) {
    throw new Error('Não foi possível verificar sua sessão.');
```

```

    return response.json();
  }

export async function login(request: LoginRequest): Promise<AuthUser> {
  const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify(request)
  });

  if (!response.ok) {
    throw new Error('E-mail ou senha inválidos.');
```

```

  }

  return response.json();
}

export async function logout(): Promise<void> {
  await fetch('/api/auth/logout', {
    method: 'POST',
    credentials: 'include'
  });
}
```

Esse cliente mantém o cookie sob controle do navegador e do servidor. O React pede o usuário atual, envia credenciais e solicita saída, mas não armazena token em `localStorage` ou `sessionStorage`. O cookie é transportado pelo navegador em chamadas compatíveis com a origem e o proxy de desenvolvimento.

O estado global de autenticação fica em `src/auth/AuthContext.tsx`, no projeto `PetMatch.Frontend`. Esse contexto concentra usuário atual, carregamento inicial e operações de entrada e saída.

```

import { createContext, useContext, useEffect, useMemo, useState } from 'react';
import { AuthUser, getCurrentUser, login as loginRequest, logout as
logoutRequest } from '../api/authClient';

type AuthContextValue = {
  user: AuthUser | null;
  loading: boolean;
  isAuthenticated: boolean;
  login: (email: string, senha: string, returnUrl?: string) => Promise<void>;
  logout: () => Promise<void>;
  refreshUser: () => Promise<void>;
};

const AuthContext = createContext<AuthContextValue | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<AuthUser | null>(null);
  const [loading, setLoading] = useState(true);
```

```
async function refreshUser() {
  const currentUser = await getCurrentUser();
  setUser(currentUser);
}

async function login(email: string, senha: string, returnUrl?: string) {
  const authenticatedUser = await loginRequest({ email, senha, returnUrl });
  setUser(authenticatedUser);
}

async function logout() {
  await logoutRequest();
  setUser({ isAuthenticated: false, email: null, nomeCompleto: null, roles:
[], claims: [] });
}

useEffect(() => {
  refreshUser().finally(() => setLoading(false));
}, []);

const value = useMemo(() => ({
  user,
  loading,
  isAuthenticated: user?.isAuthenticated === true,
  login,
  logout,
  refreshUser
}), [user, loading]);

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const context = useContext(AuthContext);

  if (context === null) {
    throw new Error('useAuth deve ser usado dentro de AuthProvider.');
```

Esse contexto evita repetição. A barra de navegação, a tela de perfil, a tela de login e o componente de rota protegida passam a consultar a mesma fonte de verdade sobre a sessão. Quando o usuário entra ou sai, a interface inteira reage ao novo estado.

O arquivo `src/main.tsx`, no projeto `PetMatch.Frontend`, passa a envolver a aplicação com `AuthProvider`, mantendo o `BrowserRouter` já usado pelo roteamento.


```
createRoot(document.getElementById('root')!).render(  
  <StrictMode>  
    <BrowserRouter>  
      <AuthProvider>  
        <App />  
      </AuthProvider>  
    </BrowserRouter>  
  </StrictMode>  
);
```

Essa composição garante que todas as rotas React tenham acesso ao estado de autenticação. A aplicação mantém a navegação por URLs e acrescenta contexto de sessão para as páginas que precisam dele. O componente `ProtectedRoute.tsx`, em `src/components`, protege rotas do front-end. Ele usa o contexto de autenticação para decidir se renderiza a página ou redireciona o visitante para `/entrar`.

```
import { Navigate, Outlet, useLocation } from 'react-router-dom';  
import { useAuth } from '../auth/AuthContext';  
import { RequestStatus } from '../RequestStatus';  
  
export function ProtectedRoute() {  
  const { isAuthenticated, loading } = useAuth();  
  const location = useLocation();  
  
  if (loading) {  
    return <RequestStatus title="Verificando acesso" description="Estamos  
confirmando sua sessão." />;  
  }  
  
  if (!isAuthenticated) {  
    return <Navigate to="/entrar" state={{ returnUrl: location.pathname }}  
replace />;  
  }  
  
  return <Outlet />;  
}
```

Esse componente melhora a navegação da SPA. A rota `/perfil` fica protegida na experiência React e a tentativa de acesso anônimo leva à tela de entrada. A proteção de dados continua no servidor e o front-end organiza a experiência antes que o usuário chegue a uma tela inadequada.

As rotas da aplicação são declaradas em `src/App.tsx`, no projeto `PetMatch.Frontend`. As páginas públicas permanecem acessíveis e o perfil fica dentro da rota protegida.

```
import { Route, Routes } from 'react-router-dom';  
import { AppNav } from '../components/AppNav';  
import { ProtectedRoute } from '../components/ProtectedRoute';  
import { LoginPage } from '../pages/LoginPage';  
import { PetDetailsPage } from '../pages/PetDetailsPage';  
import { PetListPage } from '../pages/PetListPage';  
import { ProfilePage } from '../pages/ProfilePage';  
import { RegisterPage } from '../pages/RegisterPage';
```

```
export function App() {
  return (
    <>
      <AppNav />

      <Routes>
        <Route path="/" element={<PetListPage />} />
        <Route path="/pets/:id" element={<PetDetailsPage />} />
        <Route path="/entrar" element={<LoginPage />} />
        <Route path="/criar-conta" element={<RegisterPage />} />

        <Route element={<ProtectedRoute />>
          <Route path="/perfil" element={<ProfilePage />} />
        </Route>
      </Routes>
    </>
  );
}
```

Esse roteamento organiza a SPA em páginas de produto. A vitrine e os detalhes continuam públicos. Entrada e criação de conta são páginas próprias. O perfil exige sessão autenticada. A navegação superior passa a refletir essas possibilidades. A tela de login fica em `src/pages/LoginPage.tsx`, no projeto `PetMatch.Frontend`. Ela usa o `AuthContext`, envia credenciais à API e redireciona o usuário depois de entrar.

```
import { FormEvent, useState } from 'react';
import { useLocation, useNavigate } from 'react-router-dom';
import { useAuth } from '../auth/AuthContext';

export function LoginPage() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');
  const [error, setError] = useState('');

  async function handleSubmit(event: FormEvent) {
    event.preventDefault();
    setError('');

    try {
      const returnUrl = location.state?.returnUrl ?? '/perfil';
      await login(email, senha, returnUrl);
      navigate(returnUrl);
    } catch {
      setError('Não foi possível entrar com os dados informados.');
```

```
    }
  }

  return (
    <main className="account-shell">
      <section className="account-card">
        <h1>Entrar no PetMatch</h1>
        <p>Acesse sua conta para acompanhar seu perfil e gerenciar cadastros autorizados.</p>
      </section>
    </main>
  );
}
```

```

    <form onSubmit={handleSubmit} className="account-form">
      {error && <p className="status-card status-error">{error}</p>}

      <label htmlFor="email">E-mail</label>
      <input id="email" value={email} onChange={event => setEmail(event.
target.value)} />

      <label htmlFor="senha">Senha</label>
      <input id="senha" type="password" value={senha} onChange={event =>
setSenha(event.target.value)} />

      <button type="submit">Entrar</button>
    </form>
  </section>
</main>
);
}

```

Essa tela mostra login como interação do usuário, com mensagem amigável em caso de falha. Quando o login ocorre a partir de uma tentativa de abrir `/perfil`, o retorno preserva o destino original. O mesmo padrão é usado quando o usuário tenta acessar uma ação protegida por meio dos links da interface.

O resultado visual esperado é uma SPA com barra superior sensível à sessão. Visitantes veem Pets, Cadastrar pet, Entrar e Criar conta. Usuários autenticados passam a ver Perfil e Sair. A rota `/perfil` redireciona visitantes para `/entrar`; login e cadastro usam a paleta React; o perfil mostra dados da conta, roles e claims principais; o logout retorna à aplicação ao estado anônimo. A vitrine e o detalhe dos pets permanecem públicos e as páginas protegidas do ASP.NET Core reconhecem a sessão criada pela SPA.



Resumo

O tópico 5 trata da integração entre front-end moderno e API como uma separação coordenada de responsabilidades: a interface passa a concentrar a experiência visual, a interação e a resposta imediata aos eventos do navegador, enquanto o back-end preserva contratos HTTP, validações, persistência e regras de aplicação, permitindo que as camadas evoluam com maior autonomia. Nesse contexto, React é apresentado como biblioteca baseada em componentes e estado, Node.js como ambiente de apoio ao desenvolvimento front-end, `npm` como gerenciador de dependências, TypeScript como recurso para explicitar contratos de dados e Vite como ferramenta de desenvolvimento e empacotamento adequada para consumir uma API existente sem introduzir complexidade excessiva.

O tópico também explica CORS e proxy de desenvolvimento, mostrando que origens diferentes exigem permissão explícita do back-end e que o proxy simplifica chamadas como `/api/...`, evitando acoplamento direto a portas e URLs internas. Na sequência, aprofunda o consumo de APIs com `fetch` ou bibliotecas como `Axios`, a organização de clientes HTTP fora dos componentes visuais, o tratamento de estados de carregamento, sucesso, vazio e erro, e o uso de roteamento no React para estruturar páginas de lista e detalhe, consolidando uma arquitetura em que a navegação do usuário depende de componentes, estados e URLs, mas continua apoiada em contratos HTTP estáveis fornecidos pelo servidor.

O tópico 6 apresenta autenticação e autorização como dimensões centrais da segurança ponta a ponta em aplicações web, distinguindo autenticação como verificação de identidade e autorização como decisão sobre o que o usuário identificado pode fazer, sem confundi-las com validação de dados. Em seguida, o tópico introduz o ASP.NET Identity como infraestrutura para usuários, senhas, roles, claims, tokens internos, cookies e persistência via Entity Framework Core, explicando a diferença entre autenticação por cookies, mais adequada a sessões web no navegador; e JWT, mais comum em APIs consumidas por clientes independentes, integrações e aplicações móveis. A exposição também destaca a importância da ordem do pipeline, em que a aplicação deve autenticar antes de autorizar, para que atributos como `[Authorize]` funcionem corretamente. Na parte voltada à SPA, vimos a autenticação como estado global da interface, abordando login, logout, verificação de sessão, rotas protegidas, tratamento de carregamento e erro, além da preferência por cookies protegidos com atributos como `HttpOnly`, `Secure` e `SameSite`, evitando expor tokens a `localStorage` ou `sessionStorage`.



Exercícios

Questão 1. Uma rede de bibliotecas desenvolveu uma plataforma para consulta de obras. O back-end foi implementado em ASP.NET Core e executa, durante o desenvolvimento, em `http://localhost:5213`. Ele oferece o endpoint paginado `GET /api/obras`, o endpoint de detalhe `GET /api/obras/{id}` e páginas administrativas tradicionais, como `/Obras/Create` e `/Obras/Edit/{id}`.

A interface de consulta foi criada com React e TypeScript. O ambiente Node.js executa o Vite em `http://localhost:5173`. O arquivo de configuração do Vite encaminha para o back-end os caminhos `/api`, `/Obras`, `/css` e `/js`. O ASP.NET Core também possui uma política de Cross-Origin Resource Sharing (CORS) restrita à origem do Vite no ambiente de desenvolvimento.

O cliente HTTP do front-end utiliza endereços relativos, como `/api/obras?page=1&pageSize=6`. A aplicação React emprega `BrowserRouter` e tem duas rotas:

```
<Routes>
  <Route path="/" element={<ObraListPage />} />
  <Route path="/obras/:id" element={<ObraDetailsPage />} />
</Routes>
```

A página de listagem mantém estados para os filtros, a página atual, os dados, o carregamento e os erros. Um `useEffect` refaz a consulta quando a página ou algum filtro é alterado. Os cards usam `Link` para abrir `/obras/{id}` dentro do React e um link HTML comum para acessar `/Obras/Edit/{id}` no ASP.NET Core.

Durante uma revisão, a equipe discutiu as funções do Node.js, do proxy, do CORS e dos dois mecanismos de roteamento.

Assinale a alternativa que apresenta a interpretação tecnicamente adequada dessa arquitetura.

- A) O Node.js e o Vite permanecem como ferramentas do front-end, enquanto o ASP.NET Core processa dados e regras. Como as chamadas relativas passam pelo proxy, a política de CORS pode ser removida, inclusive para eventuais requisições diretas de `http://localhost:5173` para `http://localhost:5213`.
- B) O proxy pode encaminhar `/api` e as páginas administrativas ao ASP.NET Core. Como o navegador acessa inicialmente o Vite, o `BrowserRouter` também precisa declarar uma rota `/Obras/Edit/`; sem essa declaração, o link HTML não poderá abrir a página MVC devolvida pelo back-end.
- C) A rota `/obras/` fornece ao componente o identificador da obra e permite localizar o item no estado mantido pela listagem. Assim, o endpoint `GET /api/obras/{id}` torna-se dispensável, inclusive quando o usuário abre diretamente a página de detalhes ou atualiza o navegador.

D) O Node.js executa as ferramentas de desenvolvimento do front-end e o ASP.NET Core continua responsável pelos dados e pelos endpoints. O proxy encaminha os caminhos configurados, o CORS regula chamadas diretas entre origens, o React Router seleciona componentes no navegador e o roteamento do servidor atende à API e às páginas administrativas.

E) Como o proxy encaminha `/api/obras` ao back-end, o React Router deve declarar uma rota correspondente a `/api/obras` para receber o JSON e entregá-lo ao componente. Sem essa rota de interface, a resposta produzida pelo ASP.NET Core não poderá ser processada pelo fetch.

Resposta correta: alternativa D.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: uma chamada encaminhada pelo proxy é apresentada ao navegador como uma comunicação com a origem do Vite e, nesse percurso, não depende de CORS. Uma chamada direta para outra porta possui origem diferente e somente será disponibilizada ao JavaScript se o servidor aceitar essa origem segundo sua política.

B) Alternativa incorreta.

Justificativa: o link do React Router realiza navegação interna sem recarregar o documento. Um elemento HTML comum inicia uma nova navegação no navegador. Nesse caso, o Vite pode encaminhar `/Obras/Edit/{id}` ao ASP.NET Core, que devolve a página MVC completa, sem participação de uma rota React equivalente.

C) Alternativa incorreta.

Justificativa: o estado da listagem existe apenas enquanto aquela execução da interface o conserva. Uma abertura direta, uma atualização ou uma navegação iniciada fora da listagem pode ocorrer sem esse estado. O endpoint de detalhe fornece uma fonte recuperável e independente para carregar os dados do identificador solicitado.

D) Alternativa correta.

Justificativa: a arquitetura possui dois processos e dois sistemas de roteamento. O roteador do navegador escolhe o componente da SPA, mas não consulta o banco. O roteamento do servidor escolhe o handler HTTP que produzirá dados ou páginas. O proxy apenas encaminha determinadas requisições durante o desenvolvimento e mantém as URLs do back-end fora dos componentes.

E) Alternativa incorreta.

Justificativa: as rotas do React Router controlam navegações destinadas à renderização de componentes. O fetch recebe a resposta HTTP diretamente e entrega o conteúdo ao código que iniciou a chamada. Uma requisição de dados não precisa corresponder a uma rota declarada no BrowserRouter.

Questão 2. Uma universidade desenvolveu uma plataforma para gestão de laboratórios. A aplicação possui um back-end ASP.NET Core com ASP.NET Identity e uma interface React executada como SPA (Single Page Application).

A consulta de laboratórios e equipamentos é pública. O cadastro e a edição de equipamentos exigem autenticação e uma das roles Técnico ou Administrador. A página /perfil da SPA também exige usuário autenticado. O sistema utiliza um cookie de autenticação configurado com atributos de proteção e oferece os seguintes endpoints:

- POST /api/auth/login, para validar as credenciais e emitir o cookie;
- GET /api/auth/me, para informar à SPA se existe uma sessão válida;
- POST /api/auth/logout, para encerrar a sessão;
- PUT /api/equipamentos/{id}, protegido por autenticação e role adequada.

O cliente React utiliza fetch com credentials: "include". Nas requisições que alteram o estado da aplicação, a SPA também envia, em um cabeçalho próprio, um token antifalsificação emitido pelo servidor e o back-end valida esse token antes de executar a operação. Um componente ProtectedRoute verifica o estado retornado por /api/auth/me: enquanto a sessão é consultada, apresenta uma mensagem de carregamento; se o usuário estiver anônimo, redireciona para /entrar; caso esteja autenticado, exibe o conteúdo solicitado.

Considerando a proteção ponta a ponta dessa plataforma, assinale a alternativa correta.

- A) O cookie pode ser protegido contra leitura direta pelo JavaScript e enviado por meio de credentials: "include". O ProtectedRoute controla a navegação, mas a proteção efetiva permanece no servidor, que valida o token antifalsificação nas operações de escrita, autentica antes de autorizar, aplica roles ou claims e diferencia 401 de 403.
- B) Os atributos HttpOnly, Secure e SameSite tornam desnecessário o token antifalsificação, pois impedem que outro site envie requisições com o cookie. A política de CORS também bloqueia qualquer formulário externo antes que a solicitação alcance o endpoint protegido.
- C) Como um cookie HttpOnly não pode ser lido pelo JavaScript, credentials: "include" deve ser omitido. A SPA pode enviar a role obtida em /api/auth/me por meio de um cabeçalho e o servidor pode utilizar esse valor para decidir se a edição será permitida.
- D) Se /api/auth/me devolver as roles do usuário, o ProtectedRoute pode impedir a exibição das páginas inadequadas. Nesse caso, o endpoint PUT precisa exigir apenas autenticação, pois a verificação da role já terá sido executada pela interface antes da chamada.

E) Um token antifalsificação válido comprova a identidade do solicitante e pode substituir o cookie de autenticação no endpoint PUT. A ausência desse token deve produzir 403, mesmo que nenhuma identidade tenha sido reconstruída pelo middleware de autenticação.

Resposta correta: alternativa A.

Análise das alternativas

A) Alternativa correta.

Justificativa: o cookie permite que o servidor reconstrua a identidade sem expor a credencial ao código da página. Essa identidade somente pode ser usada pelas políticas depois da execução do middleware de autenticação. O token antifalsificação atende a outro problema: demonstrar que uma operação que altera dados foi iniciada pelo fluxo esperado da aplicação. A interface controla a experiência de navegação, mas qualquer cliente pode chamar diretamente o endpoint.

B) Alternativa incorreta.

Justificativa: HttpOnly reduz a exposição do cookie a scripts, Secure restringe seu transporte a conexões protegidas e SameSite limita determinados envios entre sites. Esses atributos contribuem para a proteção, mas não tornam a validação antifalsificação dispensável em toda configuração. O CORS controla o acesso à resposta pelo JavaScript e não bloqueia, por si só, todas as formas de envio de uma requisição.

C) Alternativa incorreta.

Justificativa: o navegador pode enviar um cookie HttpOnly mesmo sem permitir que o JavaScript leia seu conteúdo. Em chamadas nas quais as credenciais não são incluídas automaticamente, credentials: "include" autoriza esse transporte. Informações de role fornecidas pelo cliente não devem ser tratadas como prova de permissão.

D) Alternativa incorreta.

Justificativa: o componente pode ocultar uma página ou redirecionar o usuário, mas seu código é executado no ambiente controlado pelo cliente. A mesma requisição pode ser produzida por outra ferramenta. A política de autorização precisa ser aplicada ao endpoint antes da execução da alteração.

E) Alternativa incorreta.

Justificativa: o token antifalsificação vincula a requisição a um fluxo legítimo, mas não substitui a credencial usada para identificar o usuário. Primeiro, o servidor precisa estabelecer a identidade; depois, deve verificar a permissão exigida. A ausência de autenticação corresponde a 401, enquanto 403 pressupõe uma identidade autenticada sem autorização suficiente.